게임 프로그래머 최재혁

포트폴리오

연락처 | 010.4787.5323 **메일** | hypernagox@tukorea.ac.kr

목차

01 프로필

(1) 소개

(2) 이력

02 졸업작품

(1) 소개

(2) 서버 구조 및 특징

- TLS + Cluster Update Queue
- Lock Free Executor
- 컴포넌트 패턴
- 패킷 핸들링

(3) 성능 향상을 위한 노력

- Zero Byte Recv
- MPSC 큐
- Custom shared_ptr
- Task vs std::function
- Cluster 공간분할
- view list (시야처리)
- Session Pooling

(4) 기타 구현내용

03 네트워크게임프로그래밍 텀프로젝트

(1) 소개

(2) 서버 구조

(3) 프로토콜 설계

(4) 구현 내용

04 기타 프로젝트

(1) Terraria-Replica(테라리아 모작)

(2) Terraria-Replica + 서버연동

소개

최 재 혁 / Choi Jae Hyuk

돌다리도 두들겨보고 건너는
멀티스레드 프로그래머

01 프로필

생년월일 1995.03.20
연락처 010-4787-5323
메일 hypernagox@tukorea.ac.kr
주소 성남시 분당구

최종학력

2013.08 검정고시 합격
2026.02 한국공학대학교 (구 한국산업기술대학교) 게임공학과 졸업예정

02 기술

언어
C
C++20

서버
IOCP
WSAPoll (Select)
Multithread

클라이언트
Win32API
OpenGL

ETC
MSSQL
Git
AWS

03 수상

2023 게임공학과 과제전 준우승[1)]
2024 게임공학과 과제전 우승[2)]
2025 전국SW중심대학우수작품경진대회 본상 수상[3)]
한국산업기술기획평가원 원장상 수상[3)]
졸업작품 최우수작품 선정[3)]
SW캡스톤경진대회 우수상 수상[3)]

1) 윈도우 프로그래밍 과목 텀프로젝트 'Terraria-Replica'
2) 컴퓨터 그래픽스 과목 텀프로젝트 'Cave Game'
3) 졸업작품 'Evergreen'

이력

학력

2025

- **한국공학대학교 게임공학과 졸업**
  2026.02 졸업예정

2024

2023

2022

- **한국공학대학교 게임공학과 입학**
  2022.03

활동

컴퓨터그래픽스 조교
2024.09 ~ 진행 중

윈도우프로그래밍 조교
2025.03 ~ 2025.08

WARP 학술소모임 C++ 멘토
2022.09 ~ 2023.12

프로젝트

Evergreen
2024.02 ~ 2025.07

Cave Game
2024.10 ~ 2024.12

Terraria-Replica IOCP
2024.05 ~ 2024.07

Terraria-Replica
2023.05 ~ 2023.09

Terraria-Replica
사이드 뷰 2D 어드벤처 샌드박스 게임
https://github.com/hypernagox/TermProjectGameFrameWork

개발 인원
서버 1명, 클라이언트 1명

개발 기간
2023.05 ~ 2023.09

Terraria-Replica IOCP
사이드 뷰 2D 온라인 어드벤처 샌드박스 게임
https://github.com/hypernagox/TUKGameServerTermProject

개발 인원
서버 1명, 클라이언트 1명

개발 기간
2024.05 ~ 2024.07

Cave Game
1인칭 온라인 어드벤처 샌드박스 게임
https://github.com/hypernagox/NetworkGameProgramming-TermProject2024

개발 인원
서버 1명, 클라이언트 1명

개발 기간
2024.10 ~ 2024.12

Evergreen
IOCP 기반 인스턴스 던전 MMORPG
https://github.com/hypernagox/FYP_Evergreen

개발 인원
서버 1명, 모델러 1명, 클라이언트 1명

개발 기간
2024.02~2025.07

졸업작품 : 소개

게임명	에버그린
게임 장르	MMORPG
개발 기간	2024.02~2025.07
개발 인원	3명 (서버, 모델러, 클라이언트)
사용 기술 (서버 기준)	C++20, IOCP, MSSQL, FlatBuffers, Python, DB Query Server , AWS
역할 및 구현 내용	서버 프로그래머 - 컴포넌트 패턴 기반의 서버프레임 워크 - 플레이어 이동, 회전, 공격, 피격 등 각종 동기화 작업 - 행동트리 기반의 몬스터 AI (행동트리를 위한 별도 프레임워크 제작) 및 FSM 기반의 몬스터 AI - 유저 간 파티 시스템 - 파티 퀘스트 컨텐츠 및 인스턴스 던전 시스템 - 다른 상용 게임과 유사한 채널 시스템 - 네비게이션 메쉬를 이용한 지형보정 및 길 찾기 - 전투, 제작, 채집, 리스폰 등을 포함한 모든 컨텐츠 시스템 - 클라이언트 사이드에서 다른 유저의 위치 정보 보간 시스템 - 클라이언트 사이드에서 서버로부터 전달되는 유저·몬스터·각종 컨텐츠 정보를 동기화 - FlatBuffers 라이브러리를 이용한 프로토콜 , 패킷 정의 및 자동화 툴 제작 - DB Query Server 제작 및 DB를 사용한 사용자 정보 관리
Git	https://github.com/hypernagox/FYP_Evergreen
시연 영상	https://www.youtube.com/watch?v=YUXPkwEod1Q
특징	SW중심대학 우수작품 경진대회 본상 수상 한국공학대전 한국산업기술기획평가원 원장상 수상 한국공학대학교 게임공학과 최우수 졸업작품 선정 한국공학대전 체육관 오프라인 전시 SW캡스톤경진대회 우수상 수상

졸업작품 : 서버 구조

1. TLS + Cluster Update Queue

- 개요:

모든 스레드가 전체 공간에 대한 자료구조를 복사해 가지고 있게 하고, 특정 공간에 이벤트 발생 시 큐를 이용하여 해당 사실을 모든 스레드에게 알리고 각자 자신의 복사된 자료구조에 해당 사실을 업데이트 하게 한다. (모든 복사된 자료구조는 같은 값을 유지)

- 나는 왜 이렇게 하였는가?:

이렇게 함으로써 특정 공간에 대한 검색 및 순회 시 해당 스레드 소유의 자료구조만을 접근하면 되기 때문에 Lock을 완전히 제거해버릴 수 있다. 따라서, Move, Attack 패킷 등을 포함한 다양한 패킷을 주변 플레이어에게 Send할 때의 순회 오버헤드가 크게 감소하며, 논타겟 투사체 등의 충돌검사를 위한 순회를 할 때에도 Lock없이 싱글스레드 로직처럼 작성이 가능하다. 그리고, 파티신청 / 귓속말 / 공격 등 Object ID를 통한 검색이 필요할 때에도 당연히 Lock이 필요 없어진다.

- 적용 전 : 가장 많이 하는 작업 인데도 불구하고 매우 비효율적으로 했어야 한다. (대부분 Lock을 걸고 복사하고 다시 순회하기 ..)

```
//void 주변에 패킷전달(패킷)
//{
// vector<shared_ptr<세션>> vec;
// lock();
// for(const auto& session:컨테이너)
// {
// // 세션이 해제되는걸 막기 위해 shared_ptr 복사
// // 이때 shared_ptr의 복사비용(레퍼런스 카운터 증가에 atomic 연산) 발생
//  vec.emplace_back(session);
// }
// unlock();
// for(const auto& session: vec)
// {
//  // 이후 여기서 패킷전달을 위한 순회를 다시..
// }
//}
```

```
//shared_ptr<오브젝트> 검색(id)
//{
//  shared_ptr<오브젝트> obj={}; // std::optional을 사용할 수도 있다.
//  lock();
//  const auto iter = container.find(id);
// // [] 이용시 없으면 빈 원소 하나 생성 / contains(id) 메서드 이용시 불필요한 검색 두번 발생
//  if(container.end() != iter)
//      obj = iter->second;
//  unlock();
//  return obj;
//}
```

```
//void 충돌검사()
//{
// vector<shared_ptr<오브젝트>> vec;
// lock();
// for(const auto& object:컨테이너)
// {
// // 오브젝트가 해제되는걸 막기 위해 shared_ptr 복사
// // 이때 shared_ptr의 복사비용(레퍼런스 카운터 증가에 atomic 연산) 발생
//  vec.emplace_back(object);
// }
// unlock();
// for(const auto& object: vec)
// {
//  // 이후 여기서 충돌검사를 위한 순회를 다시..
// }
//}
```

해당 수도코드들에서 Lock를 전부 제거할 수 있을 뿐만 아니라, shared_ptr의 복사에 대한 오버헤드 및 불필요한 순회까지 없어진다.
만약 검색해야 할 컨테이너가 선형탐색이 필요한 경우라면 더 큰 개선을 기대할 수 있다.

졸업작품 : 서버 구조

1. TLS + Cluster Update Queue

- 이걸 어떻게 구현했는가?:

1. 먼저 모든 스레드가 클러스터의 업데이트 내역 공유를 위한 중앙 큐를 제작하였다. 이때 성능을 위해 고정배열 기반 원형 Lock Free 큐로 제작하였다.

2. 각 스레드마다 현재까지 자신이 큐의 어디부분까지 업데이트 했는지 기억하는 인덱스를 TLS로 관리하게 하였다.

→

```
void ClusterUpdateQueue::UpdateCluster() noexcept
{
    constinit thread_local uint64_t L_CurIndex = 0;
    const auto arrTask = m_arrTask;
    for (;;) [[likely]]
    {
        const auto task_ptr = arrTask + ((L_CurIndex) & (MOD_TASK));
        if (const auto task = *task_ptr)
        {
            ++L_CurIndex;
            if (task->Execute())
            {
                // 한바퀴 돌면 여기서 조금 위험할 수는 있음
                *task_ptr = nullptr;
                _Compiler_barrier();
                const auto field_ptr = task->m_field;
                NagiocpX::xdelete<ClusterUpdateTask>(task);
                field_ptr->DecRef<Field>(NUM_OF_THREADS);
            }
        }
        else return;
    }
}
```

↑실제로 구현한 내용의 일부

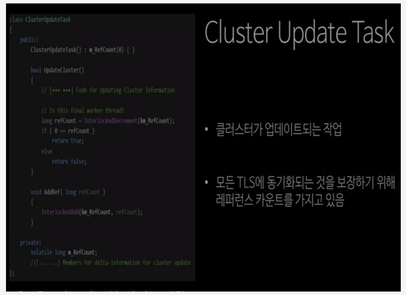
(전체 코드는 ClusterUpdateQueue.h , ClusterUpdateQueue.cpp 에 작성 되어있다.)

3. 발생한 이벤트로 인해 실제로 클러스터에 적용되어야 할 작업(함수포인터)과 필요한 매개변수를 가지고 있는 클래스를 직접 제작하였다.

4. 이때 모든 스레드가 해당 작업을 완료할 때 까지 이 클래스의 인스턴스는 유효해야 하므로 스레드가 이 일을 완료할 때 마다 1씩 감소시키는 레퍼런스 카운터를 멤버변수로서 두었다.

* 참고한 ppt 자료

(출처: https://www.slideshare.net/slideshow/mmorpg-106391346/106391346) 페이지 36. 40, 45, 46



→

```
class ClusterUpdateTask
{
    friend class ClusterUpdateQueue;
public:
    template<typename... Args>
    ClusterUpdateTask(const ClusterInfo info, Field* const field, Args&&... args)noexcept
        : m_info{ info }
        , m_invoker{ std::forward<Args>(args)... }
        , m_refCount{ ThreadMgr::NUM_OF_THREADS }
        , m_field{ FieldIncRef(field) }
    {}
private:
    bool Execute()noexcept;
private:
    volatile LONG m_refCount = ThreadMgr::NUM_OF_THREADS;
    const ClusterInfo m_info;
    Field* const m_field;
    const TaskInvoker m_invoker;
};

bool ClusterUpdateTask::Execute() noexcept
{
    m_invoker.InvokeTask(GetCluster(m_info, m_field));
    return 0 == InterlockedDecrement(&m_refCount);
}
```

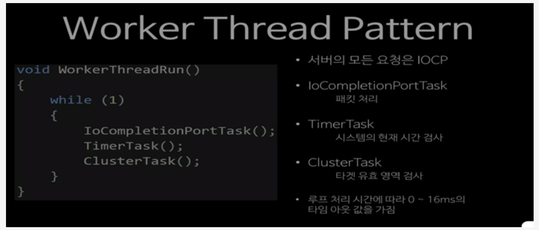
↑실제로 구현한 내용의 일부

(전체 코드는 ClusterUpdateQueue.h , ClusterUpdateQueue.cpp 에 작성 되어있다.)

5. 처음에는 참고한 ppt에서 언급된 것 처럼 TLS를 사용하여 클러스터에 접근하였으나 지역(인스턴스 던전 등)마다 클러스터 클래스의 인스턴스 개수가 달라지고 지역 생성시 소속된 클러스터의 초기화 작업에 불편함을 느껴 스레드 마다 번호를 부여하고 그 번호를 인덱스 삼아 TLS 처럼 접근하도록 설계하였다. (Cluster [스레드 번호][i][j] ...)

```
public:
    template <typename T = Cluster>
    T* const GetCluster(const Point2D sectorXY)const noexcept {
        return static_cast<T* const>(tl_vecClusters[GetCurThreadIdx()][sectorXY.y][sectorXY.x]);
    }
    template <typename T = Cluster>  <T> IntelliSense에 대한 샘플 템플릿 인수를 제공합니다.
    T* const GetCluster(const uint8_t x, const uint8_t y)const noexcept {
        return static_cast<T* const>(tl_vecClusters[GetCurThreadIdx()][y][x]);
    }
```

6. 상기한 내용을 기반으로 각 워커 스레드의 메인 루프를 발표자료를 참고하여 작성하였다.



→

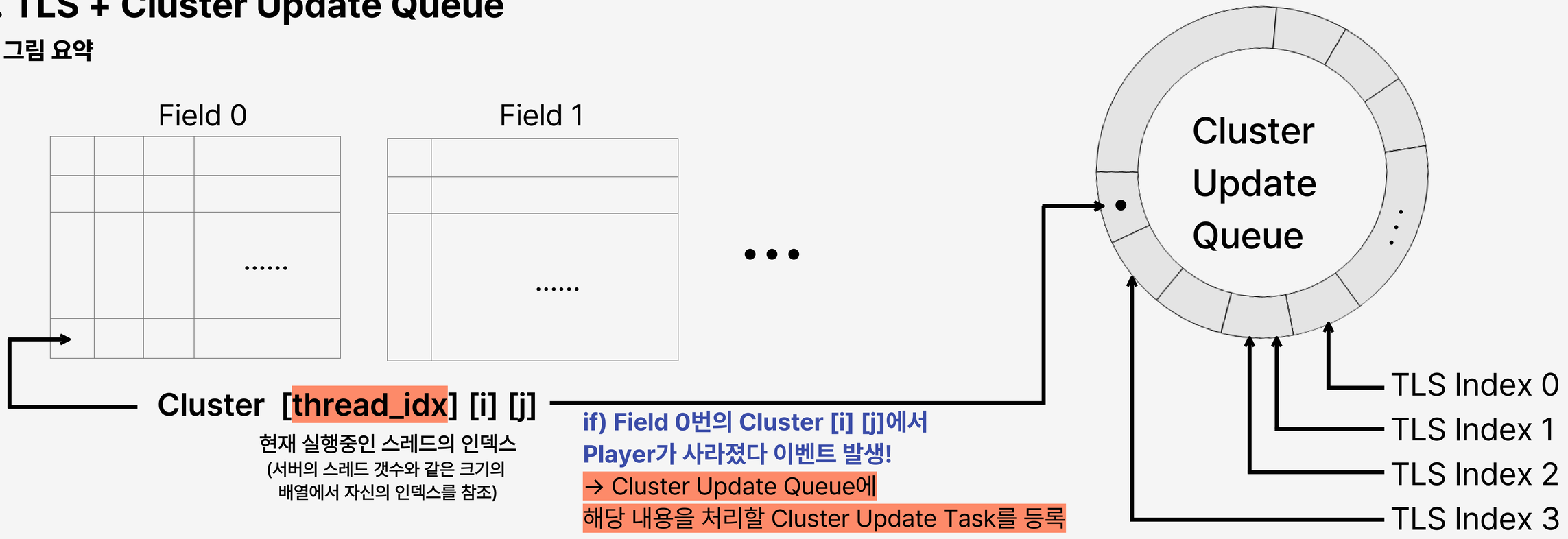
```
for(;;) [[likely]]
{
    // 1. IO 작업
    // 2. 타이머 작업
    // 3. 스레드풀에 등록된 작업
    // 4. 클러스터 업데이트 작업
    if (IocpCore::Dispatch(iocpHandle)) [[likely]]{
        taskTimer.DistributeTask();
        GlobalEventQueue::TryGlobalEvent();
        ClusterUpdateQueue::UpdateCluster();
    }
    else [[unlikely]] break;
}
```

실제로 구현한 내용의 일부 (전체 코드는 ThreadMgr.cpp에 작성 되어있다.)

졸업작품 : 서버 구조

1. TLS + Cluster Update Queue

- 그림 요약



Cluster Update Task

arg_ptr	등록된 함수포인터 실행 시 필요한 매개변수들이 있는 곳을 가리키는 포인터
fp_Task	클러스터에 발생한 이벤트를 실행할 함수포인터
Field_ptr	Task 실행 전 해당 클러스터가 소속된 Field가 할당 해제되는 것을 막기 위해 공유 포인터로 가리킴
ref_count	생성 시 서버의 스레드 개수로 초기화되는 정수값. 스레드가 이벤트를 완료하고 나면 atomic하게 1 감소시키고 감소시킨 결과값이 0이라면 모든 스레드가 해당 이벤트를 처리 완료한 것 이므로 arg_ptr이 가리키는 메모리를 해제

졸업작품 : 서버 구조

1. TLS + Cluster Update Queue

- 나는 무엇을 고민했나?:

1. 스레드0번의 Cluster에는 존재하지만 스레드1번의 Cluster에는 아직 해당 사실이 반영 안된 상태라면?

1 – 1. 스레드0 이 Object0을 클러스터에 입장시켰고 player1에게 이 사실을 알린 상태

1 – 2. player1의 화면에서는 Object0이 보이고, 이 오브젝트에게 상호작용 (귓속말, 공격, 파티신청 등)을 서버로 요청

1 – 3. 이 IO 작업을 스레드1이 받아서 처리하려 함, 근데 스레드 1의 TLS 클러스터에는 Object0이 입장했다는 사실이 아직 반영되어있지 않음

1 – 4. 스레드1의 컨테이너에 Object0이 아직 없으므로 player1의 Object0으로 의 상호작용요청을 처리할 수 없음

→ player1의 입장에서 UX적으로 에러임, 이외에도 스레드0에 의해 player1의 시야리스트에 Object0이 있었는데 스레드1이 관련로직 처리 시 컨테이너에 Object0가 존재하지 않으므로 에러 발생 가능 (현재 player의 시야리스트에는 있는데 클러스터 컨테이너에 없을 경우 해당 오브젝트가 사라졌다는 패킷을 보내고 있어서 Object0가 player1의 화면에서 사라지게 된다.)

→ 해결방안: 플레이어를 포함한 모든 오브젝트에 스레드의 개수로 초기화 된 정수 카운터를 놓고 각 스레드의 TLS 클러스터에 입장할 때마다 이 카운터를 1씩 atomic하게 감소시킨다. 이 카운터가 0인 오브젝트만 valid한 오브젝트로서 취급한다.(모든 스레드의 클러스터에 반영된 상태)

2. 클러스터에 발생한 이벤트가 큐에 등록 되어 있는 상태에서 해당 클러스터가 해제될 경우?

→ 앞서 언급한 ClusterUpdateTask 클래스에 해당 클러스터의 shared_ptr을 복사하여 ClusterUpdateTask의 소멸자가 호출되기 전 까진 Cluster의 수명을 반드시 보장하도록 함

- (정확히는 Cluster는 Field라는 공간에 소속 되어있고 Field의 인스턴스가 소멸할 경우 Cluster가 해제되는 방식이여서 Field 인스턴스의 레퍼런스 카운터를 증가시켜 수명을 보장하고 있으나 이해의 편의성을 위해 다르게 설명하였다. 정확한 코드는 ClusterUpdateQueue.h 및 cpp 참조)
- 비슷한 사례로 특정 오브젝트에 대한 이벤트를 비동기 처리 시 수명 보장을 위해 해당 오브젝트의 shared_ptr을 복사하여(레퍼런스 카운터를 올려서) 전달. (Task.h 참조)

졸업작품 : 서버 구조

1. TLS + Cluster Update Queue

- 나는 무엇을 고민했나?:

3. ClusterUpdateTask 실행 시 레퍼런스 카운터 감소를 먼저하고 실행할 경우?

3 – 1. 스레드0이 레퍼런스 카운터를 감소시켜 1이된 상태에서 컨텍스트 스위칭 발생

3 - 2. 스레드1이 레퍼런스 카운터를 감소시켜 0이된 상태에서 일을 실행 후 레퍼런스 카운터가 0이 되었으니 이 인스턴스를 해제

3 - 3. 스레드0이 컨텍스트 스위칭에서 돌아와서 이 인스턴스의 메서드를 마저 실행하려하나 이 인스턴스는 이미 메모리가 해제된 상태

→ 해결방안: 반드시 일을 먼저 수행하고 레퍼런스 카운터를 감소시킨다.

```
bool ClusterUpdateTask::Execute() noexcept
{
    m_invoker.InvokeTask(GetCluster(m_info, m_field));
    return 0 == InterlockedDecrement(&m_refCount);

    // const bool flag = 0 == InterlockedDecrement(&m_refCount);
    // 여기서 컨텍스트 스위칭 발생 시 다시 돌아왔을 때 이 인스턴스가 살아있다는 보장이 없다.
    // m_invoker.InvokeTask(GetCluster(m_info, m_field));
    // return flag;
}
```

4. ClusterUpdateTask에 실행할 함수 포인터와 필요한 매개변수를 람다 표현식으로 캡처하여 std::function으로 만들 경우 발생하는 동적할당/해제 오버헤드?

- std::function을 람다 표현식을 캡처하여 만들 경우 동적할당 발생(캡처된 변수들의 크기가 일정 바이트 이상 넘어갔을 경우)
- STL컨테이너들은 커스텀 할당자를 지정하여 메모리풀을 이용해 이 오버헤드를 감소시킬 수 있지만 std::function은 커스텀 할당자를 C++17 부터 허용하지 않음

→ 해결방안: 비슷한 역할을 하는 클래스를 만들어서 고성능 메모리 할당자를 직접 제공한다.

```
public:
    ~TaskInvoker()noexcept { m_fpTask(argPtr, nullptr); }
public:
    template<typename T, typename Ret, typename... Args>
    constexpr TaskInvoker(Ret(T::* const memFunc)(Args...), Args&&... args) noexcept
        : argPtr{ xnew<CallBack<T,Ret,Args...>>(memFunc, std::forward<Args>(args)...) }
        , m_fpTask{ [](const void* const callBackPtr_, void* const memFuncInstance) noexcept
          {
              if (memFuncInstance)
                  static_cast<const CallBack<T,Ret,Args...>* const>(callBackPtr_)->Execute(memFuncInstance);
              else
                  xdelete_sized<CallBack<T, Ret, Args...>>(static_cast<CallBack<T, Ret, Args...>*const>(const_cast<void* const>(callBa
        } }
    {
    }
public:
    inline constexpr void InvokeTask(void* const memFuncInstance)const noexcept { m_fpTask(argPtr, memFuncInstance); }
private:
    void* const argPtr;
    void (*const m_fpTask)(const void* const, void* const) noexcept;
};
```

← 직접 제작한 std::function과 비슷한 동작을 하는 클래스의 일부분 전체코드는 Task.h 참조
(MemoryMgr.h에 커스텀 할당자 존재, 이후 챕터에서 다시 자세히 설명)

졸업작품 : 서버 구조

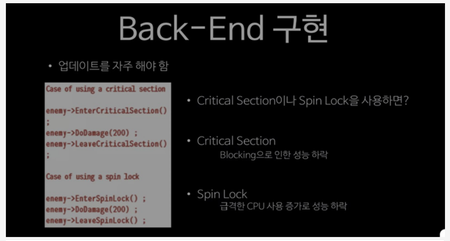
2. Lock Free Executor

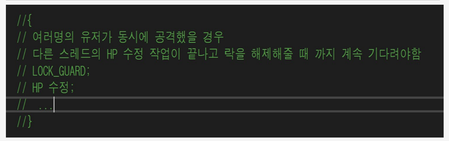
- 개요:

모든 객체에 이벤트 큐를 가지게 하여, HP 증가 및 감소 같은 원래라면 Lock을 필요로 하는 업데이트 발생 시 해당 객체의 이벤트 큐에 그 사실만을 전달하고 그 객체가 이벤트 큐에서 이벤트를 꺼내어 스스로 하나씩 업데이트 하게 한다.

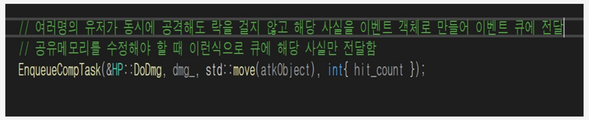
- 나는 왜 이렇게 하였는가?:

많은 유저가 몬스터를 동시에 공격했을 때 또는 많은 몬스터가 유저를 동시에 공격했을 때 mutex를 사용하여 HP변수를 수정한다면 mutex를 소유한 스레드의 HP 수정 작업이 끝날 때 까지 HP 수정 작업을 하려는 다른 모든 스레드들은 전부 기다릴 수 밖에 없고 유저와 몬스터가 많아질 수록 해당 상황은 더 심각해 진다.
이와 비슷하게 특정 객체의 공유 메모리 수정이 필요한 업데이트가 동시다발적으로 발생 시 해당 사실을 그 객체의 이벤트 큐에 전달만 하게하여 Lock을 사용하지 않고 업데이트를 가능하게 한다.





위와 비슷한 작업이 동시다발적으로 발생 시 엄청난 Blocking을 유발 ...

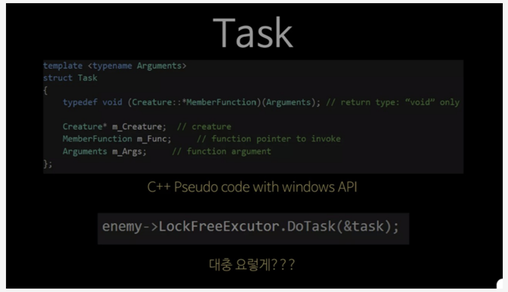


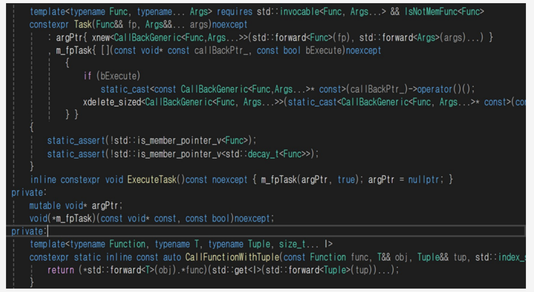
Lock Free Executor를 이용한다면 이렇게 바꿀 수 있다.

상단의 자료들은 각각 https://www.slideshare.net/slideshow/mmorpg-106391346/106391346#36 의 페이지 51, 52

- 이걸 어떻게 구현했는가?:

1. 언급한 ppt의 자료를 참고하여 실행할 작업(함수포인터)와 매개변수들을 랩핑할 수 있는 이벤트 객체(Task)를 만든다.

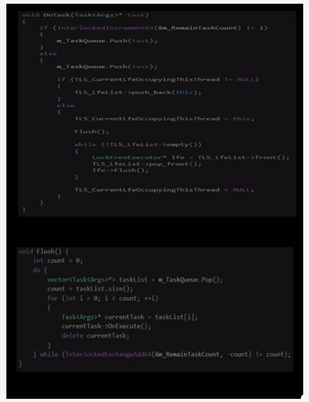


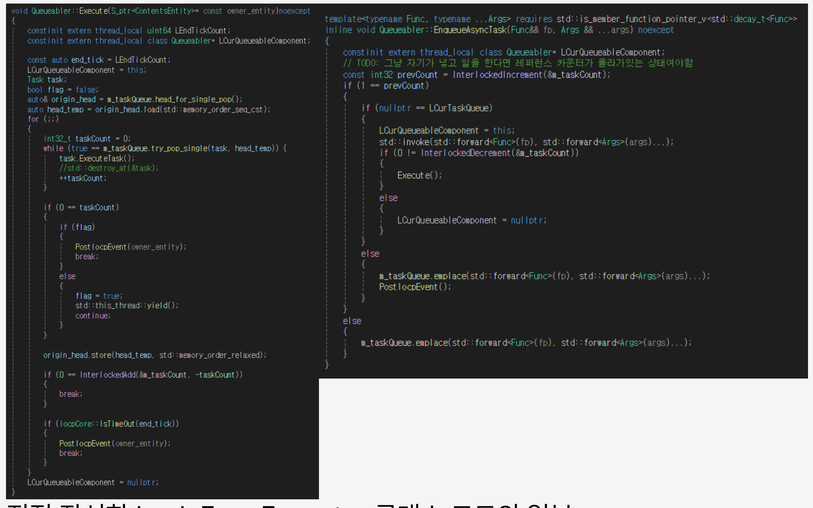


std::function과 비슷한 동작을 하면서 커스텀 할당자를 사용하는 직접 만든 Task클래스 코드의 일부(Task.h 참조)

2. 1에서 만든 Task 클래스를 원소로 가질 Lock Free Queue를 준비한다.

3. 큐의 Task를 pop하며 실제로 작업을 수행할 Lock Free Executor 클래스를 작성한다.





직접 작성한 Lock Free Executor 클래스 코드의 일부
(Queueabler 라고 이름 붙임) Queueabler.h , cpp 참조

참고한 ppt 자료 (출처: https://www.slideshare.net/slideshow/mmorpg-106391346/106391346) 페이지 55, 56

졸업작품 : 서버 구조

2. Lock Free Executor

- 나는 무엇을 고민했나?:

1. 이벤트 큐로서 사용할 Lock-Free 큐를 어떻게 할 것인가?:

- Lock-Free 큐는 여러 곳에서 제작한 자료구조가 이미 존재. 그러나 잘 생각해 보면 push 할 때는 여러 스레드가 동시에 하지만 지금 상황에서는 pop하고 일을 실행하는 건 스레드 하나로 보장된다.
- pop (delete)이 반드시 스레드 하나에서만 실행된다면 ABA문제를 신경 쓰지 않아도 된다.
- ABA문제 해결을 위한 기법(EBR, 스탬프드 노드, 하자드포인터 ...)에 발생하는 오버헤드를 없앨 수 있다.
- 이러한 내 상황에 맞는 Lock-Free큐는 찾아서 나오지 않았다.

→ 해결방안: 직접 만든다. (이 자료구조는 최적화를 위한 노력 파트에서 다시 설명)

```
}
~MPSCQueue()noexcept {
    reset();
    xdelete<Node>(head.load(std::memory_order_relaxed));
}
template <typename... Args>
void emplace(Args&&... args) noexcept {
    //const BackOff bo{ NUM_OF_THREADS / 2 };
    Node* const value = xnew<Node>(std::forward<Args>(args)...);
    // 새로운 노드를 만들고
    // 그 노드를 새 tail로 atomic하게 바꾼다.
    static_cast<Node* const>(
        InterlockedExchangePointer(reinterpret_cast<volatile PVOID*>(&tail), value)
        )->next.store(value);
    // 반환되는 값은 바꾸기 전 tail (oldTail) 이므로 해당 노드의 next에 방금 만든 tail을 연결
    //
    // 예상되는 문제점
    // tail만 바꾸고, 이전 노드의 next를 아직 바꾸지 못한 채 컨텍스트 스위칭 발생?
    // [oldTail] ->(nullptr)  ... [newTail] (아직 oldTail의 next가 nullptr을 가리키는 상태)
    // 이런 일이 여러 스레드에서 동시 다발적으로 발생한다면?
    // [oldTail 0] [oldTail 1] [oldTail 2] [oldTail 3] .... [newTail]
    // 새롭게 추가된 노드들의 next가 죄다 nullptr을 가리키는 상태
    // -> 어차피 각 스레드는 반환값으로 자신이 next를 바꿔야할 노드를 정확히 알고있음
    // 컨텍스트 스위칭에서 돌아오면 자연스럽게 next를 연결
    // pop하는 스레드는 노드의 next가 nullptr이면 empty로 간주함
    // 이 상태는 아직 큐에 삽입이 이루어지지 않았다고 생각하면 된다.
    //
    // 만약 이 상태가 마음에 들지 않아서 tail의 next를 먼저 바꾸고 그 다음 tail을 이동시킨다면?
    // 0. head와 tail이 같은 곳을 가리키는 상태에서 [head/tail]
    // 1. 스레드0이 tail의 next를 새롭게 만든 노드로 연결 시키고 컨텍스트 스위칭 됨
    //  [tail] -> [newTail] // 아직 tail은 바뀌지않음
    // 2. 스레드1이 pop하기 위해 tail(head)의 next가 nullptr이 아님을 확인함
    // 3. 스레드1이 데이터를 꺼내고 head를 head->next로 바꾸고 이전 head를 delete (head와 tail이 같은 곳을 가리키는 상태였음)
    // 4. [tail (delete 됨)] ->(???) [head] 상태가 됨
    // 5. 스레드2가 push하기 위해 tail의 next를 참조함
    // 6. 스레드2가 접근하는 tail이 가리키는 곳은 이미 스레드1이 delete한 메모리이므로 undefined behavior 발생
    //
```

직접 작성한 MPSC Queue 코드의 일부 (Multi Producer / Single Consumer의 앞글자를 따서 이름지었다.)
주석은 이때 고민한 내용이며 자세한 코드는 MPSCQueue.hpp 참조

2. 객체의 이벤트 큐에 이벤트로서 전달할 Task 클래스를 어떻게 만들 것 인가?:

- 앞서 언급한 ClusterUpdateTask와 비슷하게 std::function을 사용한다면 동적 할당/해제 오버헤드가 발생(캡처된 변수들 크기가 일정바이트 수 이상 넘어가면)
- 심지어 std::function객체 자체의 크기가 64바이트로 제법 크다.
  (비주얼 스튜디오 C++기준)

→ 해결방안: 캡처된 함수포인터와 매개변수들을 고성능 커스텀 메모리풀(je malloc 라이브러리 사용)에서 할당 받은 공간에 저장 후 해당 공간을 가리키는 void 포인터를 멤버변수로 둔다(8바이트), 그리고 해당 메모리공간을 참조하여 저장해둔 함수포인터를 실행시킬 함수포인터를 멤버변수로(8바이트) 가진 총 16바이트 크기의 Task 클래스를 직접 만든다.

```
    template<typename Func, typename... Args> requires std::invocable<Func, Args...> && IsNotMemFunc<Func>
    constexpr Task(Func&& fp, Args&&... args)noexcept
        : argPtr{ xnew<CallBackGeneric<Func,Args...>>(std::forward<Func>(fp), std::forward<Args>(args)...) }
        , m_fpTask{ [](const void* const callBackPtr_, const bool bExecute)noexcept
          {
              if (bExecute)
                  static_cast<const CallBackGeneric<Func,Args...>* const>(callBackPtr_)->operator()();
              xdelete_sized<CallBackGeneric<Func, Args...>>(static_cast<CallBackGeneric<Func, Args...>* const>(co
          } }
    {
        static_assert(!std::is_member_pointer_v<Func>);
        static_assert(!std::is_member_pointer_v<std::decay_t<Func>>);
    }
    inline constexpr void ExecuteTask()const noexcept { m_fpTask(argPtr, true); argPtr = nullptr; }
private:
    mutable void* argPtr;
    void(*m_fpTask)(const void* const, const bool)noexcept;
private:
    template<typename Function, typename T, typename Tuple, size_t... I>
    constexpr static inline const auto CallFunctionWithTuple(const Function func, T&& obj, Tuple&& tup, std::index_s
        return (*std::forward<T>(obj).*func)(std::get<I>(std::forward<Tuple>(tup))...);
    }
```

해당 클래스 구현 내용의 일부. 전체코드는 Task.h 참조
std::function에 비해 빨라진 성능은 뒤의 챕터에서 다시 언급

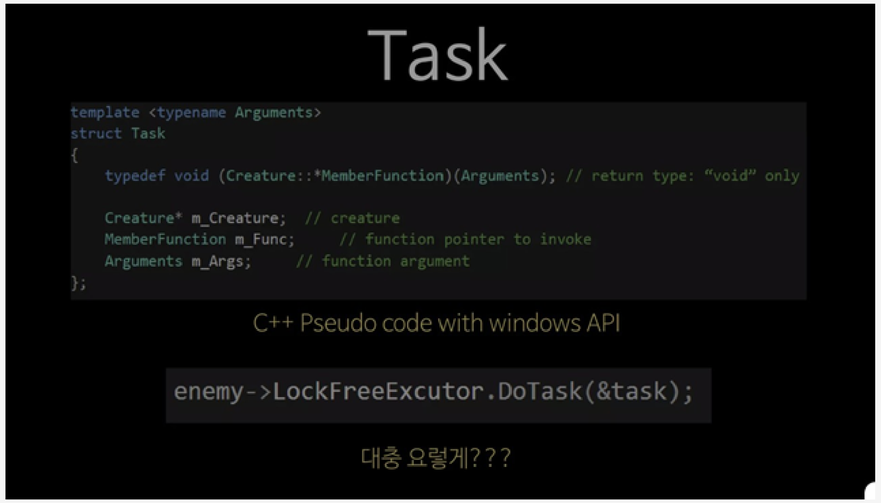
졸업작품 : 서버 구조

2. Lock Free Executor

- 나는 무엇을 고민했나?:

3. Lock Free Executor의 이벤트 큐에 처리해야 할 일이 남아있는데 LFE를 소유한 객체 (예: 몬스터, 플레이어 등...) 가 해제된다면?:

- LFE를 컴포넌트로 두고 이 컴포넌트를 소유한 오브젝트의 shared_ptr을 일감(Task 클래스) 자체에 복사하여 랩핑 한다.
- 이 때 std::move가 사용 가능한 상황이라면 이동대입연산자 또는 이동생성자를 최대한 이용하여 shared_ptr의 복사 오버헤드를 줄인다.
- 이 때 순환참조 문제가 발생할 수 있기 때문에 객체의 이벤트 큐 실행 시 반드시 flush 하도록 한다.



참고한 ppt 자료
(출처: https://www.slideshare.net/slideshow/mmorpg-106391346/106391346) 페이지 53
해당 자료에서 m_Creature 포인터가 실제 구현에서는 shared_ptr 같이 객체의 수명을 보장할 수 있는 멤버변수로 추정된다.

```cpp
void HP::PostDoDmg(const int dmg_, NagiocpX::S_ptr<NagiocpX::ContentsEntity> atkObject, const int hit_count) noexcept
{
    EnqueueCompTask(&HP::DoDmg, dmg_, std::move(atkObject), int{ hit_count });
}

void HP::PostDoHeal(const int heal_) noexcept
{
    EnqueueCompTask(&HP::DoHeal, heal_);
}

void IocpComponent::PostIocpEvent(S_ptr<ContentsEntity>* const owner_entity) noexcept
{
    auto& iocp_comp_event = GetIocpCompEvent();

    if (owner_entity && owner_entity->m_count_ptr)
        iocp_comp_event.SetIocpObject(S_ptr<IocpObject>{std::move(*owner_entity)});
    else
        iocp_comp_event.SetIocpObject(S_ptr<IocpObject>{ m_pOwnerEntity });

    GlobalEventQueue::PushGlobalEvent(&iocp_comp_event);
}

for (const auto& [obj,col] : m_obj_list)
{
    if (obj->GetPrimaryGroupType() == Nagox::Enum::GROUP_TYPE_HARVEST)continue;
    if (col->GetCollider()->IsIntersect(s))
    {
        obj->GetComp<HP>()->PostDoDmg(dmg, owner);
        isHit = true;
    }
}
```

Task 전달 시 shared_ptr 복사 및 이동과 소유 오브젝트의 수명 보장을 위한 노력의 일부분.
IocpObject.h 및 cpp 참조

졸업작품 : 서버 구조

3. 컴포넌트 패턴

- 개요:

Player, Monster, Item 구분 없이 모든 객체는 ContentsEntity이고 관련된 기능들은 전부 컴포넌트로 추가하여 상속 없이 특정 기능을 수행 하는 객체를 만들어낸다.
예를 들어 Player라면 HP, Move, Inventory 등의 클래스들을 전부 컴포넌트로 만든 뒤 해당 컴포넌트들의 조합으로 표현하고 특정 기능이 필요하다면 ContentsEntity 가 가진 컴포넌트 컨테이너에서 필요한 기능에 해당하는 컴포넌트를 찾아서 이용하는 방식

- 나는 왜 이렇게 하였는가?:

이전에는 Object 클래스를 만들고 Player를 만들기 위해 Object 클래스를 상속받은 Player 클래스를 만들고 거기서 다시 전사를 만들기 위해 Warrior 클래스를 Player를 상속받아 만들었고 몬스터의 경우도 같은 과정으로 만들었다. 하지만 이럴 경우 Object 클래스에 사소한 변경이라도 발생 했을 경우 모든 자식클래스에 영향을 미치고 또한 Player 클래스에 이동 로직이 있었는데 Monster에도 이동 로직이 필요했으나 Player를 상속받아 Monster를 만들기엔 불필요한 로직을 너무 많이 상속받아야 했고 몬스터 중에는 이동하지 않거나 다르게 이동하는 몬스터도 존재했기에 상속의 단점을 크게 체감하였고 그래서 컴포넌트 기반설계라면 해당 기능을 전부 모듈화 하여 필요한 부분만 얼마든지 재사용이 가능하다고 생각하였다.

- 이걸 어떻게 구현하였나?:

ContentsEntity 클래스에 컴포넌트들을 조합하여 특정한 객체를 생성하게 하였다.

```
S_ptr<ContentsEntity> EntityFactory::CreateRangeMonster(const EntityBuilder& b) noexcept
{
    const auto monster_entity = CreateContentsEntity(b.group_type, b.obj_type);
    monster_entity->AddComp<PositionComponent>();
    const auto fsm = monster_entity->AddIocpComponent<TickTimerFSM>();
    fsm->SetTickInterval(500);

    fsm->AddState<RangeMonIdle>();
    fsm->AddState<RangeMonChase>();
    fsm->AddState<RangeMonAttack>();

    fsm->SetDefaultState(RANGE_MON_STATE::IDLE);

    const auto agent = monster_entity->AddComp<NaviAgent>();
    agent->SetPosComp(monster_entity->GetComp<PositionComponent>());
    agent->InitRandPos(NAVI_MESH_TYPE::MAIN_WORLD);

    monster_entity->AddComp<PathFinder>()->SetAgent(agent->GetAgentConcreate());
    //monster_entity->AddComp<Collider>()->SetBox(monster_entity->GetComp<PositionComponent>(), { 1,1,1 });
    monster_entity->AddComp<SphereCollider>()->SetSphere(monster_entity->GetComp<PositionComponent>(), 1.5f);
    monster_entity->GetComp<SphereCollider>()->GetCollider()->m_offSet.y += 1.f;

    monster_entity->AddComp<HP>()->InitHP(3);
    monster_entity->AddComp<MonsterDeath>();

    // TODO: 필요할때만 딜리터 설정하기
    //monster_entity->SetDeleter<Regenerator>(5000, agent->GetPosComp()->pos);

    monster_entity->AddComp<DropTable>()->SetItemType("Bear");

    return monster_entity;
}
```

```
    }

    if (0 < m_hp)return;
    if (const auto death = owner->GetComp<Death>())
    {
        death->ProcessDeath();
    }
    if (const auto q = atkObject->GetComp<QuestSystem>())
    {
        q->PostCheckQuestAchieve(owner->SharedFromThis());
    }
}
```

필요한 기능들을 전부 컴포넌트로 만들고 그 컴포넌트들의 조합으로 Player나 Monster 같은 객체를 만들어낸다.

Death나 HP도 컴포넌트이며 Death에 관련된 작업이 필요할 경우 Death 컴포넌트를 찾아서 작업한다.

졸업작품 : 서버 구조

3. 컴포넌트 패턴

- 나는 무엇을 고민했나?:

1. ContentsEntity가 가진 컴포넌트의 컨테이너에서 컴포넌트를 어떻게 찾을까?

- 가장 먼저 떠오른 방법은 dynamic_cast를 이용하여 검색하기

가장 간단하고 확실하지만 특정 컴포넌트를 찾아 특정 작업을 하는 일이 매우 매우 잦은데 그때 마다 컨테이너를 선형탐색 하면서 매 원소마다 dynamic_cast를 할 경우의 오버헤드가 매우 크다고 판단하였다. 또한 탐색에 적합한 std::map, std::unordered_map을 놔두고 std::vector로 탐색만 한다는 점도 마음에 들지 않았다.

```
template <typename T>   <T> IntelliSense에 대한 샘플 템플릿 인수를 제공합니다.
constexpr inline T* const GetComp()const noexcept {
    for (const auto comp : container)
    {
        if (const auto target = dynamic_cast<T*>(comp))return target;
    }
    return nullptr;
```

dynamic_cast를 이용했다면 이렇게 컴포넌트 검색을 했어야 한다.

- 컴포넌트마다 enum같은 적절한 키 값을 두기

특정 컴포넌트의 static 멤버변수로 키 값을 지정하면 컨테이너에 삽입할 때 container.emlace(Comp::KeyVal, comp_ptr) 이렇게 삽입하고 찾을 때 container.find(Comp::KeyVal) 이렇게 찾을 수 있기 때문에 적절하다고 생각하였다. 하지만 적절한 Key값을 매번 정해줘야 하는 번거로움이 존재하였다.

2. 정말 std::map / std::unordered_map이 최선인가?:

앞에서 언급한 static변수로 Key값을 이용하는 방법을 사용할 때 정말 std::map / std::unordered_map 같은 자료구조를 사용하는 게 최선일까? 라는 생각이 들었다.

왜냐하면 컴포넌트들은 정말 많이 달아봐야 20개를 넘을 것 같지 않았는데 저런 적은 수의 원소를 가진 컨테이너를 검색할 때는 그냥 배열에서 선형탐색을 해도 다른 방법과 별 차이가 없을 것 같다고 생각했기 때문이다.

또한 비주얼 스튜디오 기준 std::unordered_map은 64바이트, std::vector는 24바이트이기 때문에 std::vector가 더 적은 용량을 차지하고 std::map은 16바이트이지만 vector는 메모리에서 연속적인 자료구조이기 때문에 더 좋은 캐시히트를 기대할 수 있어서 std::vector를 사용하기로 결정하였다.

졸업작품 : 서버 구조

3. 컴포넌트 패턴

- 나는 무엇을 고민했나?:

3. 컴포넌트를 식별할 키 값의 자동화:

결국 static 변수 키 값 + vector 컨테이너 사용으로 마음을 굳혔고 컴포넌트를 고유하게 식별할 Key값을 편하게 생성하기 위해 #define 매크로를 이용하여 컴포넌트의 이름을 문자열로 해석하고 그 문자열을 해시함수를 이용해 고유한 정수 키 값을 만들어 사용하기로 하였다. 이때 이 키 값 생성의 런타임 오버헤드를 피하고자 컴파일 타임에 문자열을 해시 값으로 바꾸는 consteval 함수를 직접 만들었다.

```
    return comp_ptr;
}
template <const uint64_t COMP_ID>
constexpr inline ContentsComponent* const GetCompInternal()const noexcept {
    auto b = m_contentsComponents.data();
    const auto e = b + m_contentsComponents.size();
    do {
        if (COMP_ID == b->first)return b->second;
    } while (e != ++b);
    return nullptr;
}
```

```
consteval inline const uint64_t fnv1a_hash64(const char* const str, const uint64_t hash = 0xcbf29ce484222325u) noexcept {
    return (*str == 0) ? hash : fnv1a_hash64(str + 1, (hash ^ static_cast<uint64_t>(*str)) * 0x100000001b3u);
}

#define USE_CHECK_HASH_COLLISION

#ifdef USE_CHECK_HASH_COLLISION
#define CHECK_HASH_COLLISION(ComponentType) static inline const bool g_flagForCheckHashCollision = CheckHashCollision<ComponentType>();
#else
#define CHECK_HASH_COLLISION(ComponentType)
#endif

#define COMPONENT_ID(ComponentType) static constexpr inline const uint64_t COMP_ID = fnv1a_hash64(#ComponentType);
#define GET_COMP_TYPE_FUNC_GLOBAL(ComponentType) static inline consteval const uint64_t GetCompTypeNameGlobal() noexcept { return ComponentType::COMP_ID; }
#define GET_COMP_TYPE_FUNC_VIRTUAL(ComponentType) constexpr virtual const uint64_t GetCompType()const noexcept override { return ComponentType::COMP_ID; }

#define DECLARE_COMPONENT_TYPE(ComponentType)                                   \
    private:                                                                    \
    COMPONENT_ID(ComponentType)                                                 \
    CHECK_HASH_COLLISION(ComponentType)                                         \
    public:                                                                     \
    GET_COMP_TYPE_FUNC_GLOBAL(ComponentType)                                    \
    GET_COMP_TYPE_FUNC_VIRTUAL(ComponentType)

#define CONSTRUCTOR_CONTENTS_COMPONENT(ComponentType)                           \
    public:                                                                     \
    ComponentType(NagiocpX::ContentsEntity* const pOwner_) noexcept             \
        : ContentsComponent{ pOwner_ } {}                                       \
    DECLARE_COMPONENT_TYPE(ComponentType)

#define CONSTRUCTOR_CONTENTS_UPDATE_COMPONENT(ComponentType)                    \
    public:                                                                     \
    ComponentType(NagiocpX::ContentsEntity* const pOwner_) noexcept             \
        : ContentsUpdateComponent{ pOwner_ } {}                                 \
    DECLARE_COMPONENT_TYPE(ComponentType)
```

```
class Inventory
    :public ContentsComponent
{
public:
    CONSTRUCTOR_CONTENTS_COMPONENT(Inventory)
public:

    ~Inventory();
    QuickSlot* GetQuickSlot()noexcept { return &m_quickSlot; }
    const QuickSlot* GetQuickSlot()const noexcept { return &m_quickSlot; }
public:
    const auto GetItemsAll()noexcept { return std::span<Item*>{m_items, m_items + m_curNumOfItems}; }
    const auto GetItemsAll()const noexcept { return const_cast<Inventory*>(this)->GetItemsAll(); }
public:
    bool CraftItem(const ItemRecipeData& recipe_info)noexcept;
public:
    const auto GetNumOfItems()const noexcept { return m_curNumOfItems; }
    void DecItemStack(const int8_t item_type, const int cnt)noexcept;
public:
    Item* AddDropItem(const DropItem* const drop_item_info)noexcept;
    Item* AddItem(const int item_id, const int stack_size, const bool bSend2QueryServer = true)noexcept;
```

← 컴포넌트 작성 시 매크로를 이용해 편하게 만들 수 있게 한 실제 코드의 일부

매크로를 이용해 컴파일 타임 키 값 생성 및 그 키값을 반환하는 함수를 만들도록 하였다.

4. 혹시라도 일어날 해시충돌은 괜찮은가?:

컴파일 타임 문자열 해시함수는 인터넷을 참고하여 작성하였고, 해당 함수의 해시 키 값의 충돌 확률은 매우 낮았지만 정말 혹시라도 발생할 해시 충돌에 대비하여 모든 컴포넌트는 해당 컴포넌트의 정적 멤버변수가 초기화 되는 시점에서 해시 값이 충돌하는지 여부를 검사하도록 하였다. 코드 전문은 ContentsComponent.h 및 ComponentSystem.h 참조

```
class ContentsComponent
{
protected:
    template<typename T>
    static const bool CheckHashCollision()noexcept {
        static std::set<uint64_t> g_setForCheckHashCollision;
        static std::mutex g_checkHashCollisionMutex;
        g_checkHashCollisionMutex.lock();
        if (!g_setForCheckHashCollision.emplace(T::GetCompTypeNameGlobal()).second) {
            NagiocpX::LogStackTrace();
            std::cout << "Dectected Hash Collision !!: " << __FILE__ << 'Wn' << "LINE: " << __LINE__ << 'Wn';
            g_checkHashCollisionMutex.unlock();
            NAGOX_ASSERT(false);
            return false;
        }
        g_checkHashCollisionMutex.unlock();
        return true;
    }
public:
```

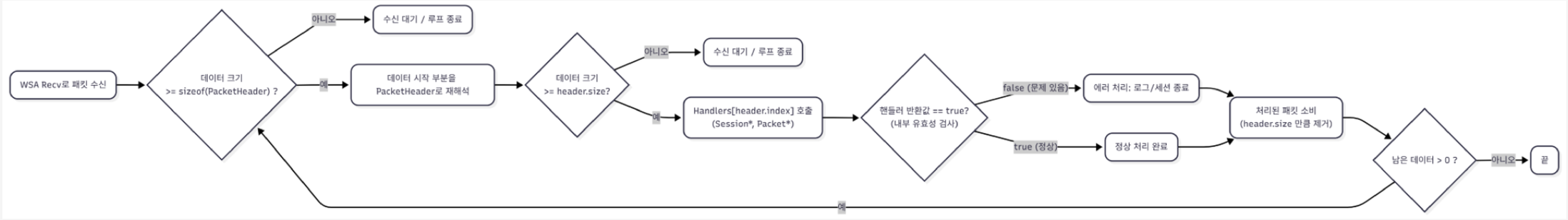
멀티스레드를 고려한 mutex 사용

졸업작품 : 서버 구조

4. 패킷 핸들링

- 개요:

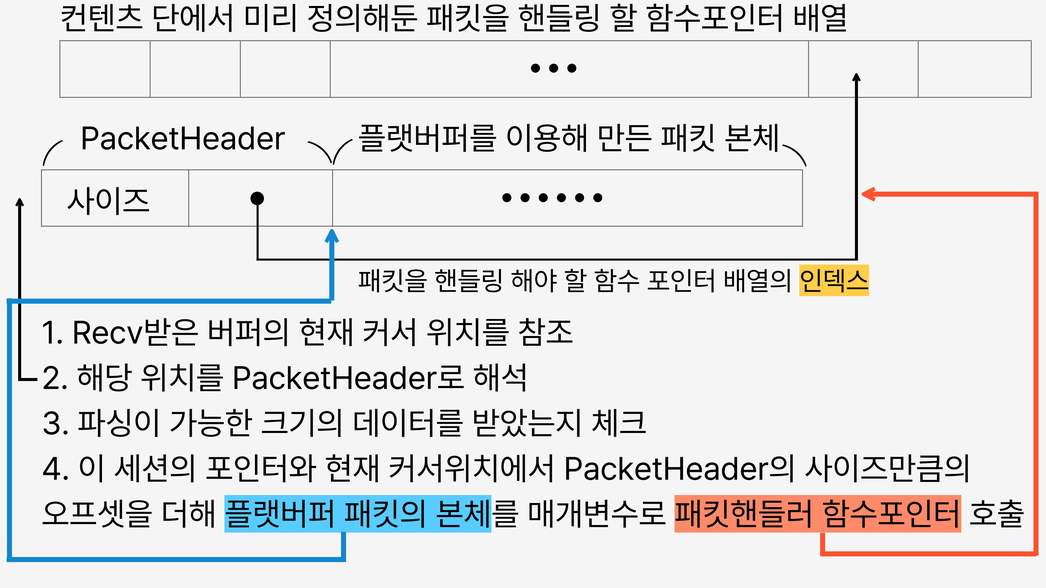
1. 컨텐츠 부분에서 특정 패킷에 대응되는 이벤트들을 정의하고 함수포인터 배열에 함수포인터로서 등록한다.
2. 해당 함수포인터 배열의 시작주소를 Session의 static 멤버변수로서 가지고 있게 한다.
3. 패킷은 해당 패킷을 핸들링 할 함수포인터 배열의 인덱스와 이 패킷의 크기정보로 구성된 PacketHeader 구조체 그리고 Flatbuffers 라이브러리를 이용한 패킷의 본체로 구성하였다.
4. 패킷 하나를 만들고 보낼 때 마다 memcpy로 복사하는 오버헤드를 피하기 위해 하나의 패킷을 shared_ptr로 할당하고 이 포인터를 주고받는 방식을 선택하였다.
5. 이렇게 만들어진 패킷이 Recv에서 도착했을 경우 ...



↑ 위의 방식은 클라이언트의 네트워크 관련 로직에도 거의 동일하게 적용하였다.

```cpp
const RecvStatus PacketSession::OnRecv(BYTE* const buffer, c_int32 len, const S_ptr<PacketSession>& pThisSessionPtr)noexcept
{
    const PacketHandleFunc* const __restrict packet_handler = g_sessionPacketHandler;
    int32_t processLen = 0;

    for(;;) [[likely]]
    {
        const PacketHeader* const __restrict header = reinterpret_cast<const PacketHeader* const>(buffer + processLen);
        const int32_t dataSize = len - processLen;

        if (dataSize < static_cast<c_int32>(sizeof(PacketHeader)))
            break;

        c_int32 packetSize = header->pkt_size;
        c_uint16 packetId = header->pkt_id;

        if (dataSize < packetSize)
            break;

        processLen += packetSize;

        // 패킷에 기입된 사이즈와 핸들링 할 패킷의 실제 사이즈가 일치하는 지 체크가 반드시 필요
        // 컨텐츠 단에서 호출할 때, 핸들링 할 패킷사이즈와 들어온 사이즈가 같은지 여부를 체크하는 부분이 있어야함
        // 패킷사이즈가 0일 때 무한루프가 돈다.
        if (!packet_handler[packetId](pThisSessionPtr, reinterpret_cast<const BYTE* const>(header), packetSize))
            return { processLen,false };
        // 나중에 이 함수는 에러코드를 반환하고, 세션의 라스트에러에 저장해서 기록하는 식으로 에러처리를 한다.
    }

    return { processLen,true };
}
```

관련 코드는 PacketSession.cpp 및 c2s_PacketHandler.h 참조



졸업작품 : 서버 구조

4. 패킷 핸들링

- 나는 무엇을 고민했나?:

1. 패킷이 뭉쳐서 오거나 쪼개져서 온다면 ?

예) 패킷 A, B가 있을 때 패킷 A의 일부만 도착한 경우 또는 패킷 A와 B를 한꺼번에 뭉쳐서 수신 받은 경우

- 해당 세션의 Recv 받은 데이터와 상황을 관리할 RecvBuffer 클래스를 제작하였다.
- 해당 클래스에는 처리를 시작해야 할 버퍼의 인덱스1과, 마지막으로 수신 받은 데이터의 바로 뒤를 가리키는 인덱스2를 가지고 있다.
- Recv를 받았다면 일단 받은 크기만큼 인덱스2를 뒤로 보낸다.
- 파싱 해보고 패킷을 처리할 수 있다면 패킷을 처리하고 인덱스1을 패킷의 크기만큼 뒤로 보낸다.
- 패킷의 일부만 도착한 경우라면 인덱스2를 뒤로만 보내서 버퍼에 기록된 데이터의 크기(인덱스2 – 인덱스1)이 패킷A의 크기만큼 될 때까지 Recv를 기다린다.
- 뭉쳐서 온 경우라면 첫번째 패킷을 처리하고 인덱스1을 뒤로 보낸 뒤 다시 남은 데이터의 크기 (인덱스2 – 인덱스1)을 확인한다.
- 남은 데이터의 크기가 다음 패킷을 처리할 수 있다면 같은 과정을 반복하고 그렇지 않다면 다시 다음 패킷을 처리할 크기가 될 때 까지 Recv를 기다린다.

```
class RecvBuffer
{
    static constexpr const inline uint32_t RECV_BUF_SIZE = 0x10000; // 64KB
    enum { BUFFER_COUNT = RECV_BUF_SIZE / PAGE_SIZE};
    enum { BUF_SLOT_SIZE = RECV_BUF_SIZE / BUFFER_COUNT };// 4KB
public:
    RecvBuffer()noexcept;
    ~RecvBuffer()noexcept { VirtualFree(m_buffer, 0, MEM_RELEASE); }
public:
    void Clear()noexcept;
    void ResetBufferCursor()noexcept { m_readPos = m_writePos = 0; }
    bool OnRead(c_int32 numOfBytes)noexcept
    {
        if (numOfBytes > DataSize()) [[unlikely]]
            return false;

        m_readPos += numOfBytes;

        return true;
    }
    bool OnWrite(c_int32 numOfBytes)noexcept
    {
        if (numOfBytes > FreeSize()) [[unlikely]]
            return false;

        m_writePos += numOfBytes;

        return true;
    }

    inline BYTE* const ReadPos() noexcept { return m_buffer + m_readPos; }
    inline BYTE* const WritePos()const noexcept { return const_cast<BYTE* const>(m_buffer + m_writePos); }
    inline c_int32 DataSize()const noexcept { return m_writePos - m_readPos; }
    inline c_int32 FreeSize()const noexcept { return m_capacity - m_writePos; }
private:
    uint32_t m_readPos = 0;
    uint32_t m_writePos = 0;
    BYTE* const m_buffer = (BYTE*)VirtualAlloc(nullptr, RECV_BUF_SIZE,
        MEM_RESERVE | MEM_COMMIT, PAGE_READWRITE);
    static constexpr inline const uint32_t m_capacity = RECV_BUF_SIZE;
};
```

← 오버랩드 구조체를 넘긴 WSA Recv는 유저가 제공한 버퍼에 페이지 단위로 PAGE_LOCKING을 걸기 때문에 PAGE단위로 버퍼를 할당하여 PAGE의 시작주소가 WSA Recv의 매개변수로 전달될 수 있게끔 유도하였다.

코드 전체는 RecvBuffer.h , cpp 참조

졸업작품 : 서버 구조

4. 패킷 핸들링

- 나는 무엇을 고민했나?:

2. Recv버퍼가 꽉 차면?:

- 앞서 언급한 실제로 처리를 시작해야 할 버퍼의 인덱스와 마지막으로 수신받은 데이터의 바로뒤를 가리키는 인덱스 (각각 인덱스1과 인덱스2)가 겹치는(같아지는) 경우 수신된 모든 데이터를 잘 처리한 것 이므로 모든 인덱스의 위치를 0부터 다시 시작하게 한다.
- 이렇지 못한 상황이 계속 발생할 경우 한계용량을 설정하고 현재 버퍼에 들어있는 처리되지 못한 데이터의 양과 설정한 한계용량의 크기를 계속 비교하여 이 크기가 한계용량을 넘어섰을 경우 처리되지 못한 데이터를 전부 0번 인덱스로 memcpy를 하고 인덱스1의 위치를 0으로, 인덱스2의 위치는 해당 데이터의 크기만큼 으로 바꾼다.

```
RecvBuffer::RecvBuffer()noexcept
    : m_readPos{ 0 }
    , m_writePos{ 0 }
{
    static_assert(0 == RECV_BUF_SIZE % (PAGE_SIZE * 16));
    static_assert(0 == RECV_BUF_SIZE % PAGE_SIZE);
}

void RecvBuffer::Clear()noexcept
{
    constexpr const int32_t HALF_SLOT = (BUF_SLOT_SIZE / 2);
    if (const int32_t dataSize = DataSize())
    {
        if (FreeSize() < HALF_SLOT)
        {
            ::memcpy(m_buffer, m_buffer + m_readPos, dataSize);
            m_readPos = 0;
            m_writePos = dataSize;
        }
    }
    else
    {
        m_readPos = m_writePos = 0;
    }
}
```

← 한계용량을 넉넉하게 설정하여 두번째 상황의 발생은 가급적 일어나지 않게 하였다.

3. PacketHeader에 들어있는 함수포인터 배열의 인덱스가 유효하지 않을 경우?:

- 컨텐츠단에서 함수포인터 배열의 크기를 UINT16MAX로 하고 이 배열을 초기화 할 때 정의되지 않은 인덱스에는 전부 아무것도 하지 않고 false를 반환하는 함수포인터를 등록시킨다.
- PacketHeader에 들어있던 인덱스 값을 UINT16으로 해석하여 배열의 범위를 초과하지 않게 한다.
- 유효하지 않은 인덱스의 함수포인터를 호출하더라도 아무것도 하지 않고 false를 반환하는 함수포인터를 호출하게 된다.

```
class c2s_PacketHandler {
    using PacketHandlerFunc = const bool (*)(const NagiocpX::S_ptr<NagiocpX::PacketSession>&, const BYTE* const, const int32_t);
    constinit static inline PacketHandlerFunc g_fpPacketHandler[UINT16_MAX] = {};
public:
for (auto& fpHandlerFunc : g_fpPacketHandler) {
    if (nullptr == fpHandlerFunc)
        fpHandlerFunc = Handle_Invalid;
}
```

4. PacketHeader에 들어있는 패킷의 사이즈가 실제 패킷의 사이즈와 다를 경우?:

- AWS 클라우드 서비스를 활용하여 게임 서버를 운영했을 때 경험한 버그
- 특히 이 값이 0일 경우 무한루프에 빠지는 심각한 문제가 발생하였음
- 실제 패킷보다 크건 작건 다음 패킷 처리 시 문제 발생 (버퍼의 엉뚱한 주소를 가리키게 됨)

→ 해결방안: 컨텐츠 단에서 실제 함수포인터를 호출하기 전 파싱한 크기와 실제 패킷의 크기가 정말 같은 지 먼저 검증하도록 한다.

```
vate:
template<HANDLE_PKT_ID packetId, typename PacketType, const bool(*const handler)(const NagiocpX::S_ptr<NagiocpX::PacketSession>&, const PacketType&
constexpr static void RegisterHandler()noexcept {
    g_fpPacketHandler[net_etoi(packetId)] = [](const NagiocpX::S_ptr<NagiocpX::PacketSession>& pSession_, const BYTE* const pBuff_, const int32_t l
        const uint8_t* const pkt_ptr = reinterpret_cast<const uint8_t* const>(pBuff_ + sizeof(NagiocpX::PacketHeader));
        flatbuffers::Verifier verifier{ pkt_ptr, static_cast<const size_t>(len_ - static_cast<const int32_t>(sizeof(NagiocpX::PacketHeader))) };
        return verifier.VerifyBuffer<PacketType>() && handler(pSession_, *flatbuffers::GetRoot<PacketType>(pkt_ptr));
    };
}
```

졸업작품 : 성능 향상을 위한 노력

성능 향상을 위해 한 것들

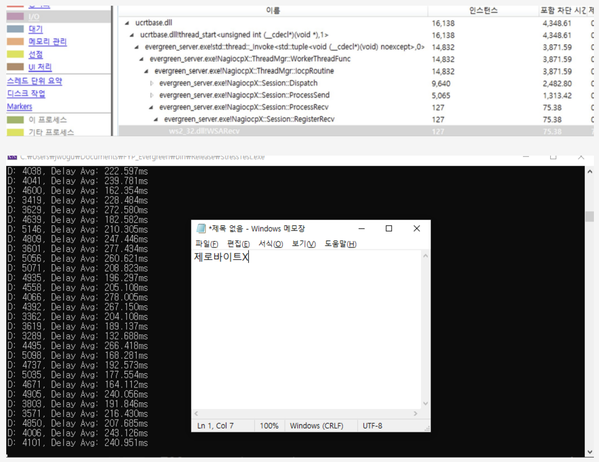
- Zero Byte Recv
- Task vs std:function
- Custom shared_ptr
- view list (시야 처리)
- Cluster 공간분할
- Session Pooling
- MPSC 큐

1. Zero Byte Recv

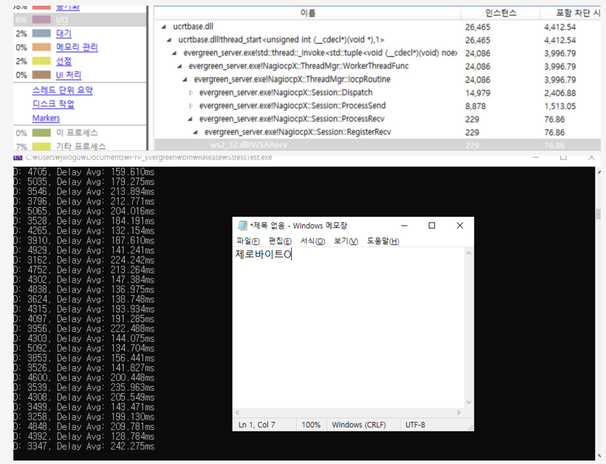
- 나는 이걸 왜 하였는가?:

- 오버랩드 구조체를 넘긴 WSARecv 호출(비동기 작업 요청)시 PAGE LOCKING을 걸게 된다.
- WSARecv 호출 시 버퍼 사이즈를 0으로 제공하면 PAGE LOCKING을 걸지 않게 된다.
- 0 바이트의 WSARecv의 비동기 작업 완료 시 곧 바로 제대로 된 버퍼 크기를 제공하여 WSARecv를 이용하여 바로 데이터를 수신
- 얻는 것: 아직 Recv 받은 게 없는 세션에 대해서 non – paged pool의 낭비 및 PAGE LOCKING의 오버헤드를 줄일 수 있게 된다.

하지만 오히려 WSA Recv의 호출횟수가 늘어나서 성능 향상이 없었다. →
오히려 같은 조건에서 Zero Byte Recv를 적용했을 때 WSA Recv에 의한 차단 시간이 더 길어졌다.



Zero Byte Recv (X)
평균 딜레이 211ms

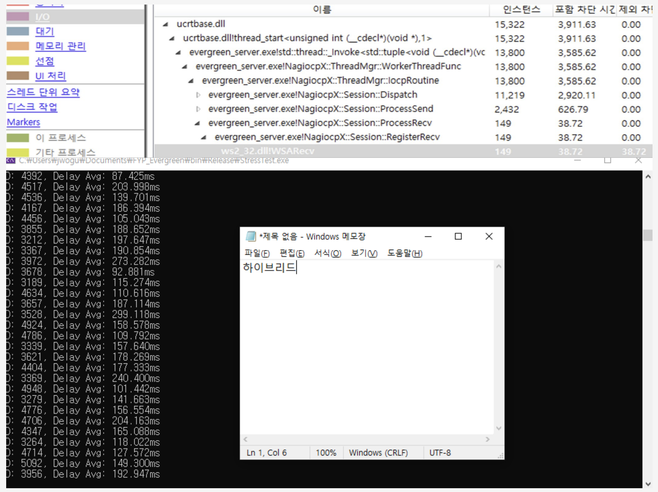


Zero Byte Recv (O)
평균 딜레이 213ms

- 패킷을 수신 받았을 때 성공적으로 모든 패킷을 처리해서 세션의 리시브 버퍼에 더 이상 처리해야 할 데이터가 없다면 다음 패킷은 약 0.5초 후에 올 것으로 기대하기 때문에 (이동 패킷 전송 주기가 0.5초) Zero Byte Recv를 사용
- 반대로 만약 리시브 버퍼에 패킷 조각이 남아 있다면 바로 나머지 패킷 조각을 수신할 수 있을 것으로 예상해서 Zero Byte Recv를 사용하지 않는 하이브리드 방식을 적용하였다.

```
m_bTurnOfZeroRecv &= (0 == recv_buff->DataSize());
```

↑패킷 처리 후, 남은 데이터가 있는지 여부를 Zero Byte Recv를 관리하는 플래그에 and연산을 적용하여 구현



← 하이브리드 방식 평균 딜레이 149ms
WSA Recv에 의한 차단 시간이
75ms~76ms에서 38ms 수준으로 감소

※ 테스트 환경
CPU: 라이젠 3600
RAM: 16G
동시접속자수 3000명
몬스터 10만 마리

졸업작품 : 성능 향상을 위한 노력

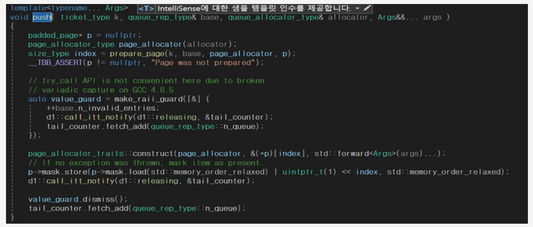
성능 향상을 위해 한 것들

2. MPSC 큐 (Multi Producer Single Consumer 큐)

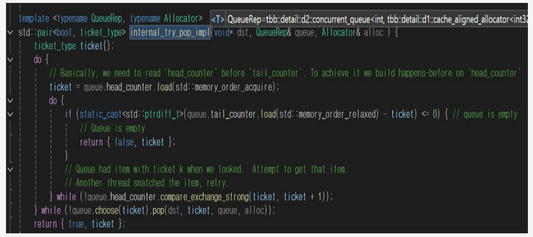
- 나는 이걸 왜 하였는가?:

앞서 언급된 Lock Free Executor의 작동 방식이 Pop (Flush)할 때 반드시 싱글스레드로 동작함이 보장되기에 ABA 문제 해결을 위한 오버헤드를 없앨 수 있고 일반적인 Lock-Free큐의 Push / Pop 로직 구현에 필요한 루프 + CAS 구조에서 루프를 제거하고 CAS연산을 더 가벼운 atomic 연산으로 바꿀 수 있기에 큰 성능 향상을 기대할 수 있다고 생각하였다.
또한 Session의 Send 로직도 앞서 언급한 방식과 매우 유사하게 작동하기 때문에 Session의 Send 큐에도 MPSC 큐를 사용한다면 이 부분에서도 마찬가지로 큰 성능 향상을 기대할 수 있다고 생각하였다.

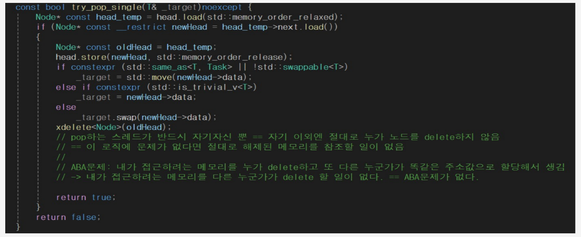
- tbb 큐 구현 vs MPSC 큐 구현



→



→



좌측이 tbb의 concurrent_queue / 우측이 직접 제작한 MPSC 큐
MPSC 큐에서는 루프가 사라지고 CAS가 다른 atomic 연산으로 대체되었다.

- tbb 큐 성능 vs MPSC 큐 성능

VS

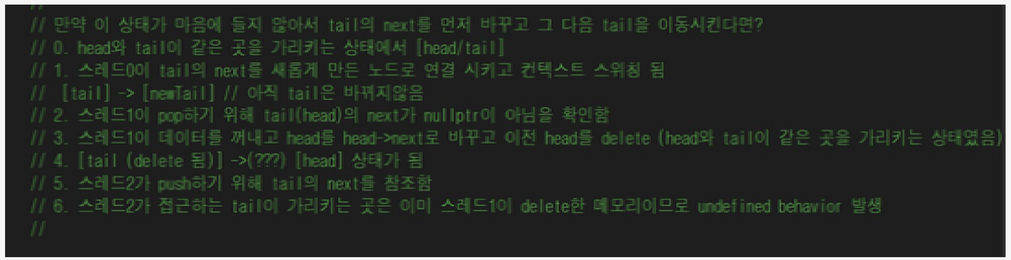
```
TBB concurrent_queue (single consumer)
 : 7685.414 ms
MPSCQueue (single consumer)
 : 6349.832 ms
```

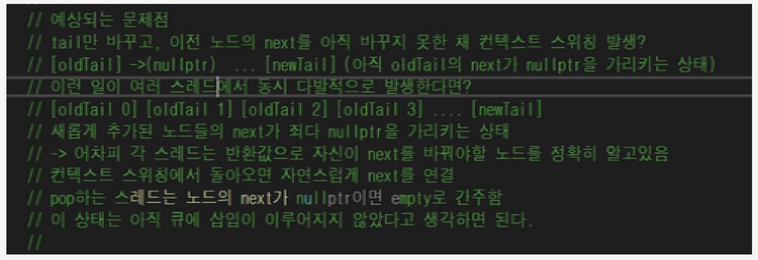
← 12스레드(생산자)가 각각 천만개의 정수를 push하며 단일 소비자가 전부 pop 했을때의 벤치마크 결과 약 1350ms 성능 향상을 보여준다.

```
TBB concurrent_queue (single consumer)
 : 74293.848 ms
MPSCQueue (single consumer)
 : 71255.780 ms
```

← 12스레드(생산자)가 각각 1억개의 정수를 push하며 단일 소비자가 전부 pop 했을때의 벤치마크 결과 약 3050ms 성능 향상을 보여준다.

→ tbb 큐에 비해 평균적으로 10% 이상의 성능향상을 확인할 수 있었다.





←자료구조 작성 시 고민했던 문제들....

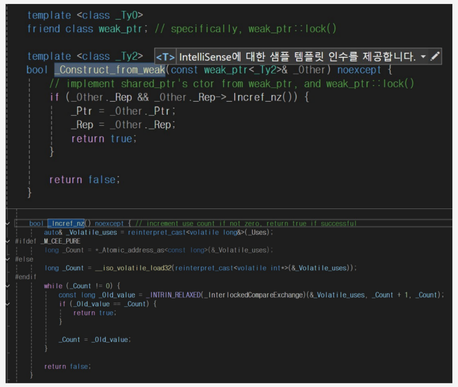
졸업작품 : 성능 향상을 위한 노력

성능 향상을 위해 한 것들

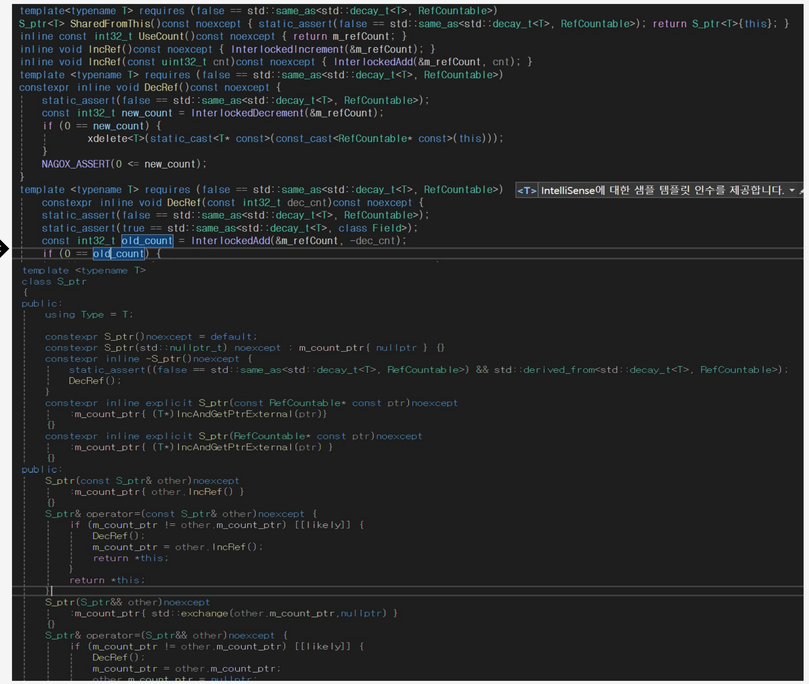
3. Custom shared_ptr 제작

- 나는 이걸 왜 하였는가?:

shared_ptr을 이용하고 있다면 this포인터를 전달할 필요가 있을 때 반드시 std::enable_shared_from_this를 상속받아서 shared_from_this()를 호출하여 생성하여야 한다.
현재 사용하는 MSVC 기준 std::enable_shared_from_this는 자기자신의 weak_ptr을 멤버변수로 가지고 이를 매개변수로 shared_ptr의 생성자를 이용하여 반환하도록 구현되어 있다.
shared_from_this는 매우 자주 호출될 함수인데 내부 구현이 루프+CAS로 되어있어 큰 오버헤드가 예상되고 직접 구현하면 InterlockedIncrement를 한번만 호출하는 것으로 성능 개선 가능성이 보인다.
현재 사용하는 MSVC 기준 shared_ptr의 크기가 16바이트인데 직접 제작한다면 이를 8바이트로 줄일 수 있고 제어블록 대신 정수 값 하나만을 이용하여 더 적은 메모리로 구현할 수 있다.



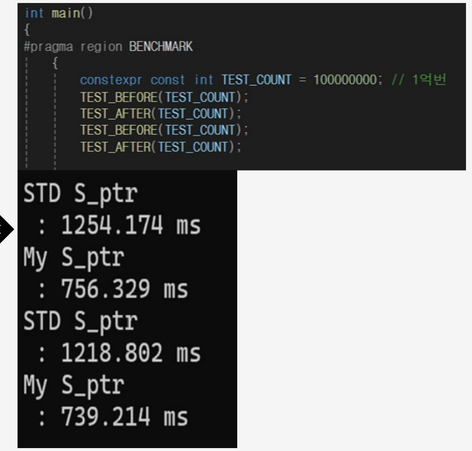
표준 shared_from_this()의 내부 구현
(MSVC 기준)



루프와 CAS를 없애고 InterlockedIncrement 호출
한번으로 간단하게 shared_from_this를 구현 가능
(상세한 코드 전문은 RefCountable.h 참조)

- Custom shared_ptr 제작으로 얻는 것:

1. 표준 shared_ptr에 비해 훨씬 더 적은 메모리로 구현 가능
2. 객체의 수명 보장을 위해 불필요한 shared_ptr의 복사가 여러 번 발생 하는 것을 한번에 레퍼런스 카운터를 1보다 더 큰 값을 증가시키는 것으로 대체 가능
3. 자기 자신에 대한 shared_ptr 생성(shared_from_this)에 발생했던 오버헤드를 크게 감소



직접만든 shared_ptr과 표준 shared_ptr의 shared_from_this를 각각 1억번씩 호출하였을때
약 500ms 빨라졌으며, 40%정도의 성능향상이 있었다

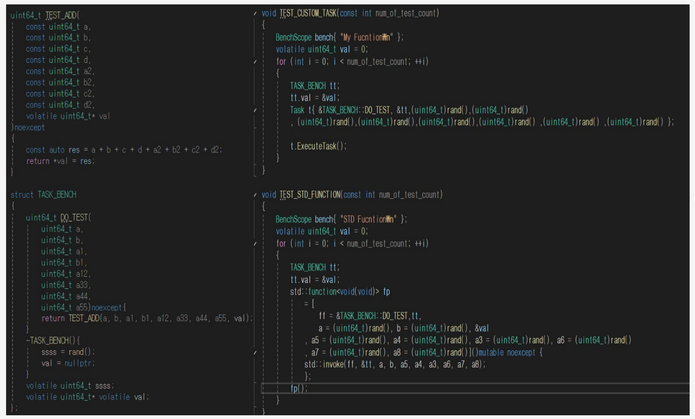
졸업작품 : 성능 향상을 위한 노력

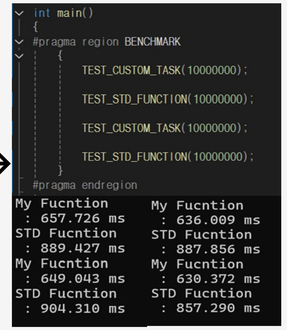
성능 향상을 위해 한 것들

4. Task vs std::function

- 나는 이걸 왜 하였는가?:

- std::function을 람다 캡처로 만들 시 발생하는 동적 할당/해제 오버헤드를 고성능 커스텀 메모리 풀을 이용하여 개선할 수 있다.
- x64 기준 16바이트 크기(포인터 두개)로 제작이 가능하여 64바이트인 std::function에 비해 훨씬 더 적은 메모리를 이용하여 구현 할 수 있다.



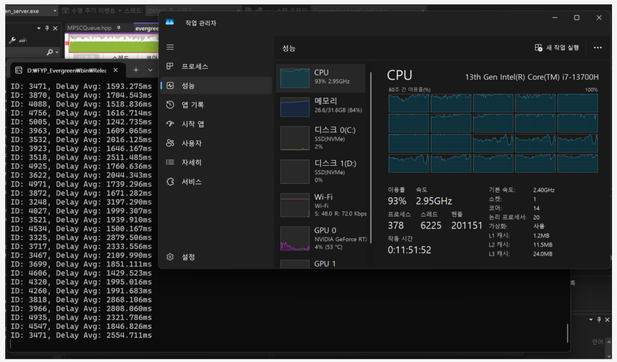


직접 만든 Task와 std::function을 동일한 멤버함수 포인터와 매개변수를 캡처하여 생성 및 각각 천만번 씩 호출했을 때 Task 클래스가 약 250ms 더 빨랐으며, 28%정도의 성능향상을 확인할 수 있었다

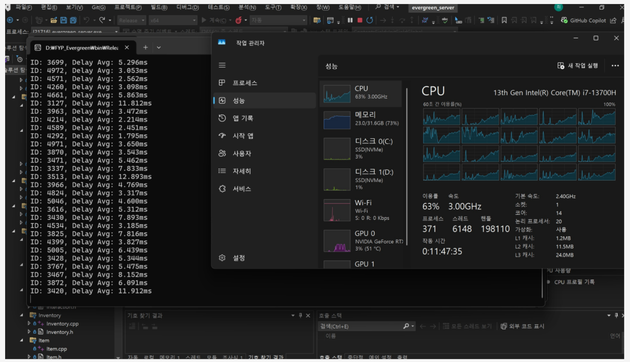
5. Cluster 공간분할

- 나는 이걸 왜 하였는가?:

패킷 전달, 충돌검사 등 순회 및 특정 조건에 맞는 오브젝트를 찾아내기 위해 선형탐색을 진행할 때매번 월드전체의 오브젝트를 검색하기엔 탐색공간이 너무나도 크다.
월드를 격자판으로 쪼개서 자신이 속한 격자판과 그 근처의 격자판만 검색한다면 탐색공간을 크게 줄일 수 있다.
게임서버의 많은 알고리즘들이 대부분 시간복잡도가 O(N^2) 수준이기 때문에 반드시 이 N을 줄여야만 한다고 생각하였다.
(이 프로젝트에서 격자판 하나를 Cluster라고 부르며, Cluster [x / cluster_x_scale] [z / cluster_z_scale]와 같은 방식으로 자신이 속한 Cluster에 접근할 수 있도록 하였다.)



공간분할 X
딜레이 약 2000ms , CPU 사용량 93%



공간분할 O
딜레이 약 10ms , CPU 사용량 63%

더미 클라이언트를 이용한 동시접속자 2000명 시뮬레이션 결과 공간분할 유무에 따른 엄청난 성능차이가 발생한다.
특히 공간분할 적용 후의 서버의 CPU는 아직도 여유가 있는 상태

(자세한 코드는 ClusterInfoHelper.h / cpp 및 main.cpp 의 main 함수 밑 GlobalClusterFilter 참조
컨텐츠 단에서 어디에서 누구를 어떻게 쪼개고 필터링 할 것인가에 대한 함수포인터를 서버엔진쪽에 전달하는 식으로 동작한다.)

졸업작품 : 성능 향상을 위한 노력

성능 향상을 위해 한 것들

6. view list (시야처리)

- 나는 이걸 왜 하였는가?:

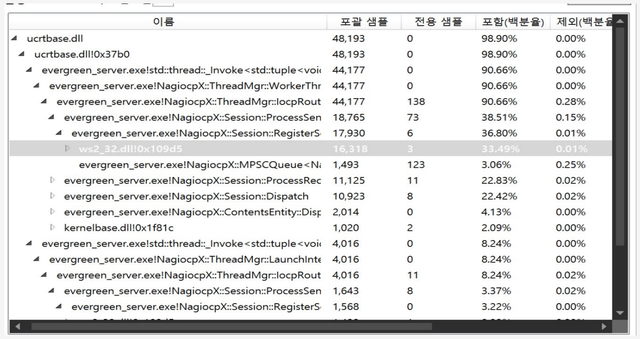
성능을 위해 Concurrency Visualizer로 프로파일링 해본 결과 Move패킷 전달 시 발생하는 WSASend의 오버헤드가 가장 문제라고 판단하였다.

하지만 WSASend함수는 내부 구현에 접근할 수 없어서 직접 최적화를 시도하는 것은 불가능하였다.

따라서 WSASend의 호출 횟수를 줄이는 방향으로 접근하였고 각 Player마다 자신의 시야에 존재하는 오브젝트들을 관리하는 자료구조를 두어 이 자료구조에 존재하는 오브젝트에 대해서만 WSASend를 호출하도록 하였다.

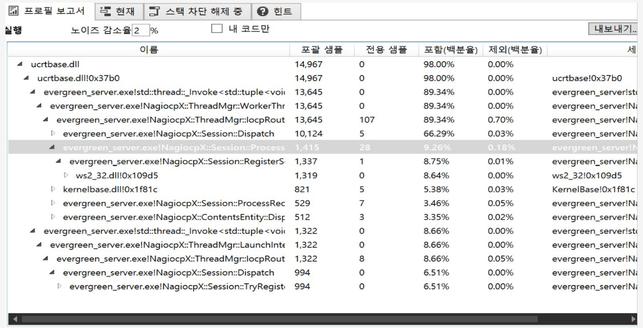
이 자료구조를 업데이트 할 때 순회와 검색이 둘 다 많이 발생하여 std::vector와 std::unordered_map을 조합해 메모리에서 연속적인 구조를 유지하면서도 검색/삽입/삭제에 O(1) 시간복잡도를 가진 자료구조를 제작하였다.

(상세한 코드는 MoveBroadcaster.h / cpp 및 VectorHash.hpp 참조)



Concurrency Visualizer를 이용한 view_list 적용 전 성능분석 결과 Send관련 함수의 오버헤드가 가장 높게 측정되었다.

시야처리 전 2천명의 더미 클라이언트 부하테스트 결과 약 160ms의 딜레이와 92%의 CPU 사용량



시야처리 적용 후 개선된 프로파일링 결과
Send관련 함수의 실행 지분이 38.51% → 9.26% 로 감소

더미클라이언트 5천명 수준에서 10ms 딜레이와 41%의 CPU 사용량

```
class VectorSetUnsafe
{
    using IndexType = uint32_t;
    using TableContainer = TableBase<Value, IndexType>;
public:
    constexpr VectorSetUnsafe()noexcept = default;
    constexpr VectorSetUnsafe(const std::size_t size_)noexcept { reserv
    void reserve(const std::size_t size_)noexcept {
        if constexpr (std::same_as<TableContainer, XHashMap<Value, Inde
            m_IDZidx.reserve(size_);
        m_listItem.reserve(size_);
    }
public:
    template <typename V>
    const bool AddItem(V&& value)noexcept
    {
        if (false == m_IDZidx.try_emplace(value, static_cast<const int3
            return false;
        m_listItem.emplace_back(std::forward<V>(value));
        return true;
    }

    const auto EraseItemAndGetIter(const Value& value)noexcept
    {
        if (const auto iter = m_IDZidx.extract(value)) [[likely]]
        {
```

```
    const bool HasItem(const Value& val)const noexcept { return m_IDZidx.contains(val); }

    constexpr const auto begin()const noexcept { return m_listItem.begin(); }
    constexpr const auto end()const noexcept { return m_listItem.end(); }
    constexpr const auto begin()noexcept { return m_listItem.begin(); }
    constexpr const auto end()noexcept { return m_listItem.end(); }

    constexpr const auto cbegin()const noexcept { return m_listItem.cbegin(); }
    constexpr const auto cend()const noexcept { return m_listItem.cend(); }
    constexpr const auto cbegin()noexcept { return m_listItem.cbegin(); }
    constexpr const auto cend()noexcept { return m_listItem.cend(); }

    constexpr const auto size()const noexcept { return m_listItem.size(); }

    const auto& GetItemListRef()const noexcept { return m_listItem; }
    auto& GetItemListRef()noexcept { return m_listItem; }

private:
    inline const int32_t GetLastIndex()const noexcept { return static_cast<const int32_t>(m_listItem.size() - 1); }
private:
    XVector<Value> m_listItem;
    TableContainer m_IDZidx;
};
```

vector와 해시테이블을 같이 이용하여 메모리에서 연속적인 구조를 유지하면서 검색/삽입/삭제에 O(1) 시간복잡도를 가진 커스텀 자료구조 (VectorHash.hpp 참조)

```
// view_list new_list;
// view_list& old_list = m_my_view_list;
// for 자신이 속한 Cluster 및 그 주변 Cluster들의 오브젝트들에 대해서 :
// if 오브젝트와 나와의 거리가 시야에 보일 정도라면 :
// new_list에 추가한다.
//  if 내 시야리스트에 이미 이 오브젝트가 있다면 :
// 그 오브젝트에게 Move 패킷을 보낸다. // 오브젝트가 세션이라면
//  else
//      // 월드에는 존재하는데 내 시야에는 없었다. == 시야에 이제 막 나타났다는 뜻
//      내 시야리스트에 이 오브젝트를 추가하고
//      나와 상대방에게 서로의 아바타정보를 포함한 Appear 패킷을 보낸다. // 오브젝트가 세션이라면
//
//      for old_list의 오브젝트들에 대해서 :
// if new_list에 이 오브젝트가 없다면 :
// // 내 시야에 있었는데 이제 없어진 것이므로
// 내 시야리스트에서 해당 오브젝트를 제거하고
//  나와 상대방에게 각각 서로에 대한 Remove 패킷을 보낸다. // 오브젝트가 세션이라면
//  // 위의 과정을 전부 거쳤다면 old_list가 곧 new_list가 된다.
```

시야리스트 업데이트 알고리즘의 수도코드
상세한 코드는 MoveBroadcaster.cpp 참조

졸업작품 : 성능 향상을 위한 노력

7. Session Pooling

- 나는 이걸 왜 하였는가?:

클라이언트가 접속할 때마다 소켓을 포함한 리시브 버퍼 등 세션에 필요한 자원들을 동적할당하고, 접속 종료 시 해당 자원들을 전부 해제해야 했었다.

게임서버 특성상 유저의 접속 및 종료는 매우 빈번하게 발생하기 때문에 서버 초기화 시 정해진 수 만큼 세션을 만든 다음 모든 소켓을 IOCP에 등록해두고 이를 계속 재사용 함으로써 소켓을 포함한 세션과 관련된 자원의 할당/해제의 오버헤드를 줄이고자 하였다.

```
template<typename T>  <T> T=NagiocpX::Session *
class MPSCStack
{
    struct Node
    {
        T data;
        NagoxAtomic::Atomic<Node*> prev;
        template<typename... Args>
        constexpr Node(Args&&... args)noexcept :data{ std::forward<Args>(args)... } {}
    };
public:
    void clear()noexcept {
        Node* node = m_top;
        m_top = nullptr;
        while (node) {
            Node* const next = node->prev;
            xdelete<Node>(node);
            node = next;
        }
    }
    ~MPSCStack()noexcept { clear(); }
public:
    template <typename... Args>
    void emplace(Args&&... args) noexcept {
        const BackOff bo;
        Node* const newTop = xnew<Node>(std::forward<Args>(args)...);
        for (;;)
        {
            Node* const oldTop = m_top;
            newTop->prev.store_relaxed(oldTop);
            if (oldTop != m_top)continue;
            if (oldTop == InterlockedCompareExchangePointer(reinterpret_cast<volatile PVOID*>(&m_top), newTop, oldTop))
            {
                return;
            }
            else
            {
                bo.delay();
            }
        }
    }
    const bool try_pop_single(T& target)noexcept {
        for (;;)
        {
            Node* const oldTop = m_top;
            if (nullptr == oldTop)return false;
            Node* const newTop = oldTop->prev.load_relaxed();
            if (oldTop != m_top)continue;
            if (oldTop == InterlockedCompareExchangePointer(reinterpret_cast<volatile PVOID*>(&m_top), newTop, oldTop))
            {
                target = std::move(oldTop->data);
                xdelete<Node>(oldTop);
                return true;
            }
        }
    }
    const bool empty_single()const noexcept { return nullptr == m_top; }
private:
    Node* volatile m_top = nullptr;
```

↑ Accept를 받는 스레드가 반드시 하나임을 보장 할 수 있어서 세션 풀에 이용할 자료구조로 MPSC 스택을 직접 작성하였다.
(Multi Producer Single Consumer Stack)

```
Session* const Service::CreateSession()noexcept
{
    auto pSession = m_sessionFactory();
    pSession->SetService(this);
    return m_iocpCore.RegisterIOCP(pSession, (uint64_t)p
}
```

```
NAGOX_ASSERT(0 < m_maxSessionCount);
NAGOX_ASSERT(nullptr != global_packet_func);
Session::InitializePacketHandleFunc(global_packet_func);
g_mainService = this;
m_id2Index.reserve(maxSessionCount_ * 2);
for (int i = m_maxSessionCount; i >= 1; --i)
{
    const auto session = CreateSession();
    NAGOX_ASSERT_LOG(nullptr != session, "Fail To Create Session");
    _Post_ _Notnull_ session;
    session->m_serviceIdx = i;
    m_sessionPool.emplace(session);
}
std::atomic_thread_fence(std::memory_order_seq_cst);
```

```
S_ptr<Session> PopSession()noexcept{
    Session* pSession;
    if (m_sessionPool.try_pop_single(pSession) ){
        return S_ptr<Session>{reinterpret_cast<const uint64_t>(pSession)};
    }
    else {
        return S_ptr<Session>{};
    }
}
```

↑ 서버 초기화 시 세션 생성 및 세션 풀에서 꺼내 쓰는 코드

```
void Service::ReturnSession(Session* const pSession) noexcept
{
    const auto session_limit = m_maxSessionCount - 1;
    PrintLogEndl("Return Session");
    pSession->ClearSessionForReuse();
    const bool re_accept_flag = (session_limit == InterlockedDecrement((LONG*)&m_curNumOfSessions));
    m_sessionPool.emplace(pSession);
    if (re_accept_flag)
    {
        if (SERVICE_TYPE::SERVER == m_eServiceType)
        {
            const auto& listener = static_cast<ServerService*>(this)->GetListener();
            if (false == listener->m_bCanAccept.exchange(true)) {
                listener->RegisterAccept(listener->m_acceptEvent.get());
                PrintLogEndl("Accept available !");
            }
        }
    }
}
```

```
template<typename T>
constexpr inline void xdelete(T* const obj_ptr)noexcept{
    static_assert(alignof(T) <= 8);
    if constexpr (std::same_as<std::decay_t<T>, ContentsEntity>)
        return obj_ptr->ProcessCleanUp();
    else if constexpr (std::derived_from<std::decay_t<T>, Session>)
        return ReturnSession(obj_ptr);
    else if constexpr (std::same_as<std::decay_t<T>, SendBufferChunk>)
        return SendBufferMgr::ReturnChunk(obj_ptr);
    else if constexpr (!std::is_trivially_destructible_v<T>)
        obj_ptr->~T();
    Memory::Free(obj_ptr);
}
```

```
template <typename T> requires (false == std::same_as<std::decay_t<T>, RefCountable>)
constexpr inline void DecRef()const noexcept {
    static_assert(false == std::same_as<std::decay_t<T>, RefCountable>);
    const int32_t new_count = InterlockedDecrement(&m_refCount);
    if (0 == new_count) {
        xdelete<T>(static_cast<T* const>(const_cast<RefCountable* const>(this)));
    }
    NAGOX_ASSERT(0 <= new_count);
}
```

↑ Custom shared_ptr을 이용한 세션 반환 코드

MPSCStack.hpp / Service.cpp / MemoryMgr.h / RefCountable.h 참조

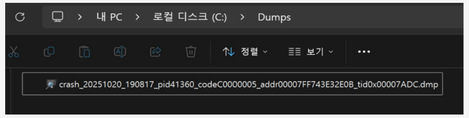
졸업작품 : 기타 구현 내용

Dump

Dump를 이용한 디버깅

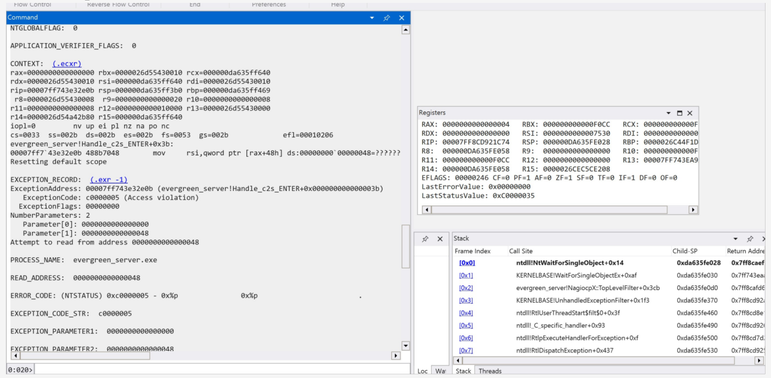
```
static BOOL WriteDumpNow(const HANDLE hFile,
    const EXCEPTION_POINTERS* const ep,
    const MINIDUMP_TYPE type,
    const DWORD crashTid)
{
    MINIDUMP_EXCEPTION_INFORMATION mei{};
    MINIDUMP_EXCEPTION_INFORMATION* pMei = nullptr;
    if (ep)
    {
        // 크래쉬 스레드의 TID
        mei.ThreadId = crashTid;
        mei.ExceptionPointers = const_cast<EXCEPTION_POINTERS*>(ep);
        mei.ClientPointers = FALSE;
        pMei = &mei;
    }

    MINIDUMP_CALLBACK_INFORMATION mci{};
    mci.CallbackRoutine = DumpCallback;

    MINIDUMP_USER_STREAM_INFORMATION usi{};
    MINIDUMP_USER_STREAM stream{};
    std::unique_ptr<char[]> owned;

    BuildUserStreams(const_cast<EXCEPTION_POINTERS*>(ep), usi, stream, owned, crashTid);

    const BOOL ok = ::MiniDumpWriteDump(
        ::GetCurrentProcess(),
        ::GetCurrentProcessId(),
        hFile,
        type,
        pMei,
        &usi,
        &mci
    );

    ::FlushFileBuffers(hFile);
    ::CloseHandle(hFile);
    return ok;
}
```

```
static void BuildUserStreams(const EXCEPTION_POINTERS* const ep,
    OUT MINIDUMP_USER_STREAM_INFORMATION& usi,
    OUT MINIDUMP_USER_STREAM& stream,
    OUT std::unique_ptr<char[]>& owned,
    const DWORD crashTid)
{
    wchar_t img[MAX_PATH]{}; ::GetModuleFileNameW(nullptr, img, MAX_PATH);
    const wchar_t* const cmd = ::GetCommandLineW();
    SYSTEMTIME st; ::GetLocalTime(&st);
    const DWORD pid = ::GetCurrentProcessId();
    const DWORD tid = crashTid; // 크래쉬 발생한 스레드 ID
    const DWORD code = ep ? ep->ExceptionRecord->ExceptionCode : 0xE0000001;
    const void* const addr = ep ? ep->ExceptionRecord->ExceptionAddress : nullptr;

    wchar_t text[2048]{};
    ::StringCchPrintfW(text, _countof(text),
        L"[Crash Summary]\r\n"
        L"Time        : %04u-%02u-%02u %02u:%02u:%02u\r\n"
        L"Process     : %s (pid=%u)\r\n"
        L"Crash Thread : tid=%u (0x%08X)\r\n" // 크래쉬 스레드 ID
        L"ExceptCode  : 0x%08X\r\n"
        L"ExceptAddr  : %p\r\n"
        L"CmdLine     : %s\r\n",
        st.wYear, st.wMonth, st.wDay, st.wHour, st.wMinute, st.wSecond,
        img, pid, tid, tid, code, addr, cmd ? cmd : L"(null)");
```



서버 크래쉬 발생 시 크래쉬가 발생한 스레드의 ID 및 여러 정보를 포함한 Dump를 생성하게 하여 해당 파일을 이용해 디버깅을 하였다.
(CrashDump.cpp 참조)

Dump를 이용한 디버깅 사례



00007ff743e32e0b 488b7048 mov rsi,qword ptr [rax+48h] ds:00000000`00000048=?????????????? 이 어셈블리를 번역해줘

다음 한 줄을 해석하면 돼요:

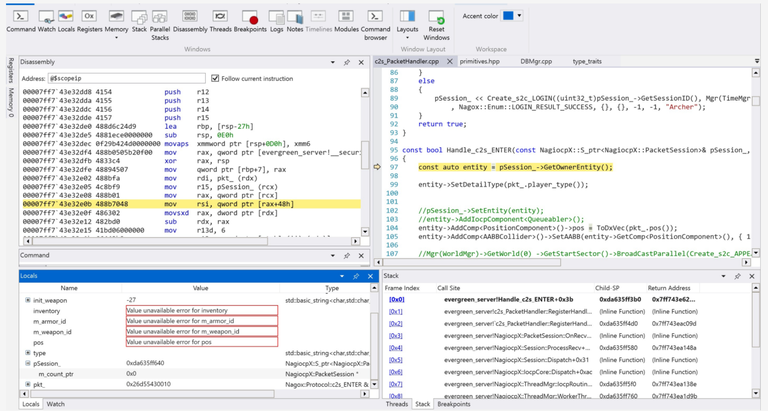
ruby 코드 복사

```
00007ff7`43e32e0b  48 8B 70 48    mov rsi, qword ptr [rax+48h]
ds:00000000`00000048 = ??????????????
```

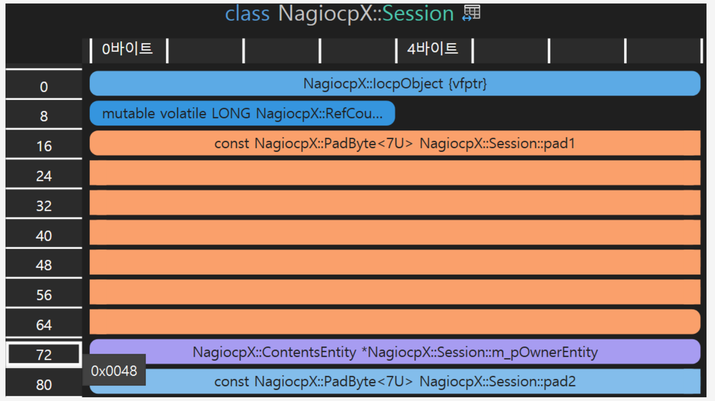
무엇을 하는 명령인가?

- **의미:** `rsi ← *(uint64*)(rax + 0x48)`
  - 즉, 레지스터 **RAX**가 가리키는 주소에서 **0x48 바이트** 떨어진 위치의 **8바이트(qword)** 값을 읽어 **RSI**에 넣습니다.

↑ 생성된 덤프의 크래쉬 발생 지점 어셈블리를 AI로 번역
rax 레지스터의 값이 0이었기 때문에 발생한 문제.



↑ 왜 rax에 0이 들어 있었는지 분석



rcx에 매개변수 pSession_의 주소가 들어있었고 rax에 pSession_객체의 첫 8바이트 값을 저장.
pSession_은 Custom shared_ptr이기 때문에 이 값은 랩핑한 포인터 값이다.
그런데 이 값이 0이었다.
결론: 랩핑한 객체의 시작 주소 + 오프셋으로 m_pOwnerEntity에 접근을 시도하였으나 Custom shared_ptr이 nullptr을 랩핑하고 있었다.

졸업작품 : 기타 구현 내용

1. DB 쿼리서버

DB관련 로직을 수행하는 서버를 따로 제작하여 메인 서버는 해당 서버의 소켓으로 관련 요청만 보내게 함으로써 메인 서버의 부하를 감소시켰다.
(첨부된 Git의 DBQueryServer 솔루션 참조)

2. 행동트리 및 FSM을 이용한 몬스터 AI

행동트리 프레임워크를 직접 제작하여 보스와 같은 복잡한 몬스터는 행동트리를, 간단한 일반 몬스터의 경우 FSM을 이용한 몬스터 AI를 구현하였다.
(TickTimer.h / cpp , BehaviorTree.hpp , ContentsBehaviorNode.h / cpp , BossBehaviorNode.h , cpp 참조)

3. 채널 및 인스턴스 던전 구조

유저를 분산시키기 위한 채널 및 파티 퀘스트 수행을 위한 인스턴스 던전을 구현하였다. (PartyQuestSystem.h / cpp , FieldMgr.h / cpp 참조)

4. 파티 퀘스트 컨텐츠

인스턴스 던전에서 수행할 파티 퀘스트 컨텐츠들을 구현하였다. (NPC 호위퀘스트, 사냥 퀘스트, 침투 퀘스트)
(QuestRoom.h / cpp 참조)

5. 파티시스템

수락, 거절이 가능하며 파티 퀘스트 수행 도중 파티장이 나간다면 그 다음 유저가 자동으로 파티장이 되는 파티 시스템을 구현하였다. (PartyQuestSystem.h / cpp 참조)

6. 움직임 보간 시스템

클라이언트 사이드에서 다른 유저의 위치 동기화를 0.1초 ~ 0.5초에 한번만 해도 자연스럽게 보이도록 보간 시스템을 구현하였다. (서버 부하 감소)
(클라이언트 쪽 솔루션의 MovePacketSender.h / cpp , MoveInterpolator.h / cpp , Interpolator.hpp 참조)

7. 클라이언트 사이드에서의 네트워크 관련 로직

클라이언트 사이드에서 연결 및 패킷 송수신 로직을 구현하고, 서버로부터 전달되는 유저·몬스터·각종 컨텐츠 정보들을 동기화하였다.
(ServerObjectMgr.h / cpp , ServerObject.h / cpp , ServerComponent.h / cpp 참조)

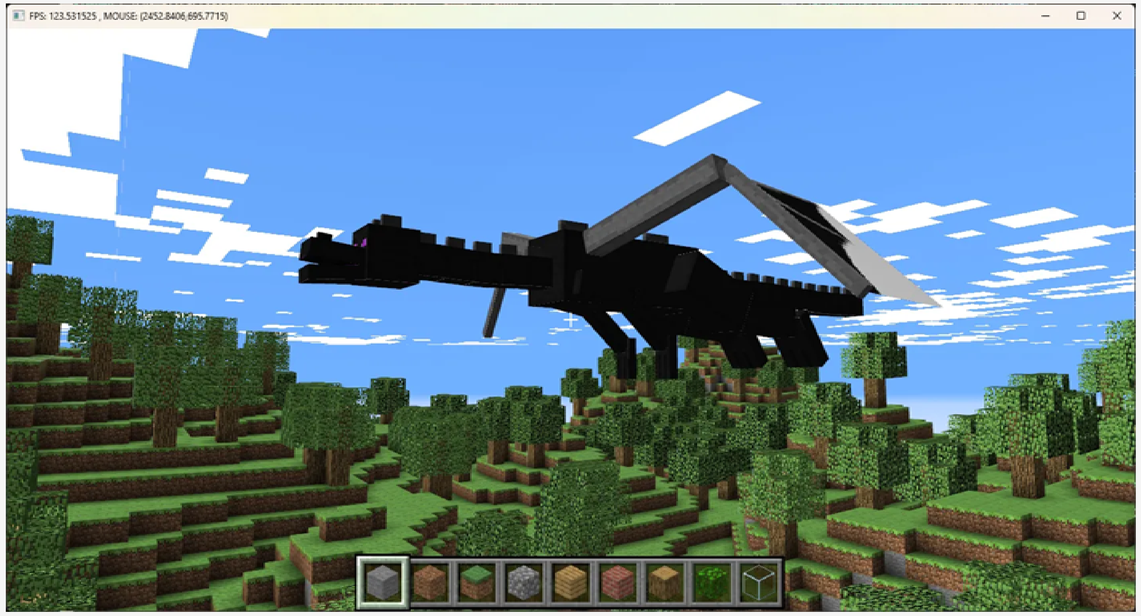
8. FlatBuffers 라이브러리 및 자동화 툴 제작

FlatBuffers 라이브러리를 이용하여 프로토콜 및 패킷을 정의하였고 이를 간편하게 사용하기 위해 파이썬과 Jinja2 라이브러리를 이용한 툴을 제작해 프로토콜 및 패킷 작성을 자동화하였다.

9. 더미 클라이언트를 이용한 부하 테스트

단순한 랜덤 움직임이 아닌 가까운 채집물들 중 k번째로 가까운 채집물을 향해 채집을 시도하고, 공격 및 스킬을 사용하여 간단한 전투가 가능한 더미클라이언트를 이용해 서버의 안정성 테스트 및 성능 측정이 가능하도록 하였다.
(StressTest 솔루션 참조)

네트워크 게임 프로그래밍 텀프로젝트 : 소개

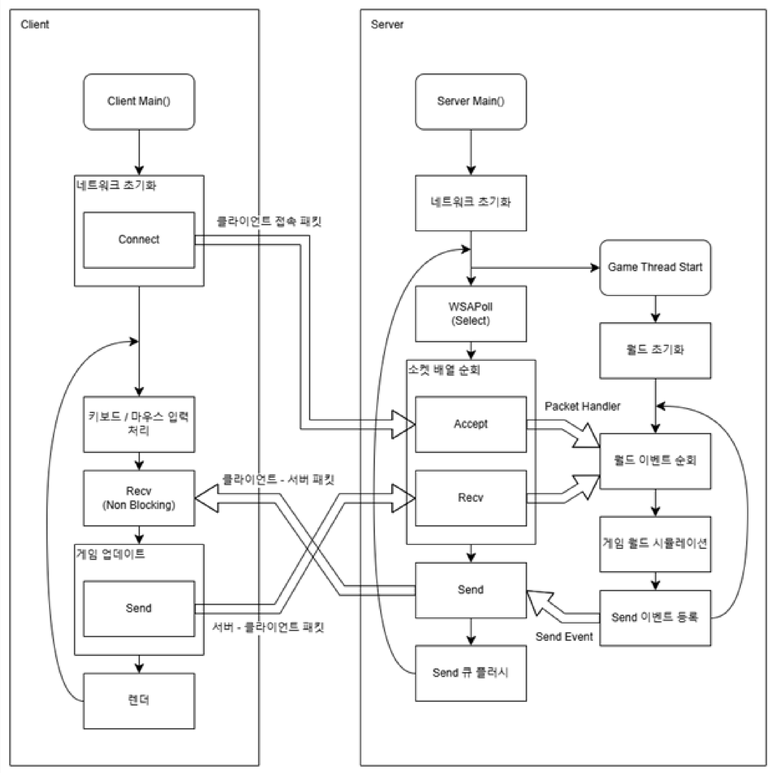


게임명	Cave Game (마인크래프트 모작)
게임 장르	1인칭 어드벤처 샌드박스 게임
개발 기간	2024.10 ~ 2024.12
개발 인원	2명
사용 기술	C++20, OpenGL, WSAPoll
역할 및 구현 내용	- OpenGL 기반 클라이언트 프레임워크 - WSAPoll 기반 서버 프레임워크 - 클라이언트 사이드에서 연결 및 패킷 송수신 로직 - 서버로부터 전달되는 유저·몬스터·각종 컨텐츠 정보를 동기화 - 프로토콜 및 패킷 설계 - FSM 기반 몬스터 AI - 3차원 배열을 이용한 A* 길 찾기 알고리즘 - 보스 몬스터 패턴 및 베지어 곡선을 이용한 이동 로직 - 유저간 입/퇴장 및 이동 동기화 - 투사체 로직 및 충돌검사 - 아이템 드랍 및 획득 로직
Git	https://github.com/hypernagox/NetworkGameProgramming-TermProject2024
노션링크	https://www.notion.so/140c647a121b80019b76c5ce137b0375?v=10fc4f4363444003ae3252645a791d0d&source=copy_link
특징	스탠드 얼론 게임으로 제작되었던 컴퓨터 그래픽스 과목 텀 프로젝트를 멀티플레이를 지원하도록 확장한 프로젝트 (멀티플레이 연동 전 시연 영상: https://www.youtube.com/watch?v=fNS8UQee1eE)

네트워크 게임 프로그래밍 텀프로젝트 : 서버구조 / 프로토콜 설계

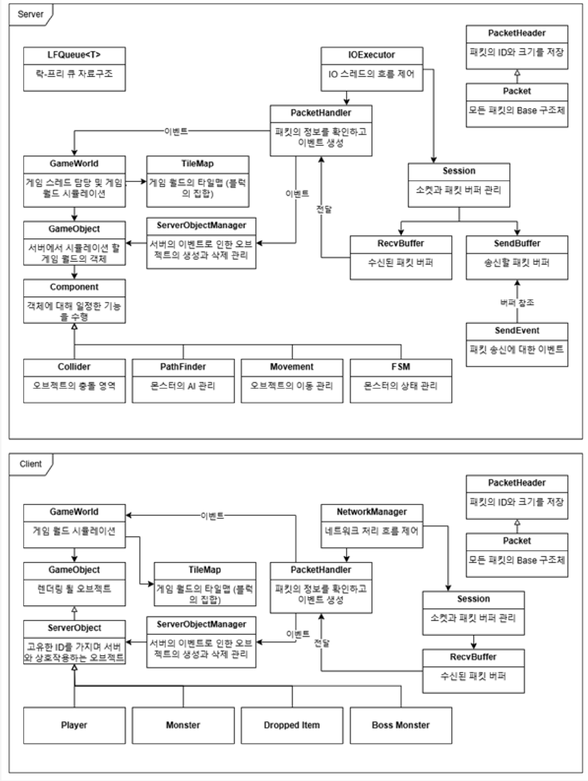
서버 구조

- IO 작업을 전담하는 IO 스레드와 게임 월드를 시뮬레이션하는 월드 스레드로 분리하여 서버 구조를 설계하였다.
- IO작업에 의해 게임월드의 변화가 발생한 경우 (예: 유저의 입/퇴장) 월드 스레드의 이벤트 큐로 해당 내용을 전달하게 하였다.
- 게임 월드에서 유저에게 Send를 해야 할 경우 바로 Send하지 않고 해당 루프에서 Send해야 할 모든 패킷을 하나의 큰 버퍼에 전부 이어 붙여서 루프 마지막에 IO스레드의 이벤트 큐로 전달만 하도록 하였다.



클라이언트 및 서버의 흐름도

Server\\main.cpp, Server\\IOExecutor.h
Server\\IOExecutor.cpp, Server\\MCWorld.h
Server\\MCWorld.cpp 참조



주요 클래스 및 기능

프로토콜 설계

```
#define DECLARE_PACKET(pkt_name)                                                    ₩
    pkt_name() : PacketHeader{sizeof(pkt_name), etoi(PKT_ID::pkt_name)} {}          ₩
    private:                                                                        ₩
        static void HandlePacket(const uint64 id, const char* const pBuff_) {      ₩
            return pkt_name::Handle(id, *reinterpret_cast<const pkt_name* const>(pBuff_));  ₩
        }                                                                           ₩
    static bool RegisterHandleFunc() noexcept                                       ₩
    {                                                                               ₩
        if constexpr ((#pkt_name[0] == 'c' && G_PROJECT == PROJECT_TYPE::SERVER) ||  ₩
                      (#pkt_name[0] == 's' && G_PROJECT == PROJECT_TYPE::CLIENT))   ₩
        {                                                                           ₩
            pkt_name temp;                                                          ₩
            AddProtocol(static_cast<unsigned short>(temp.pkt_id), &pkt_name::HandlePacket);  ₩
        }                                                                           ₩
        return true;                                                                ₩
    }                                                                               ₩
    static const inline bool g_bInitPacket = RegisterHandleFunc();                  ₩
    static void Handle(const uint64 id, const pkt_name& pkt_) noexcept;
```

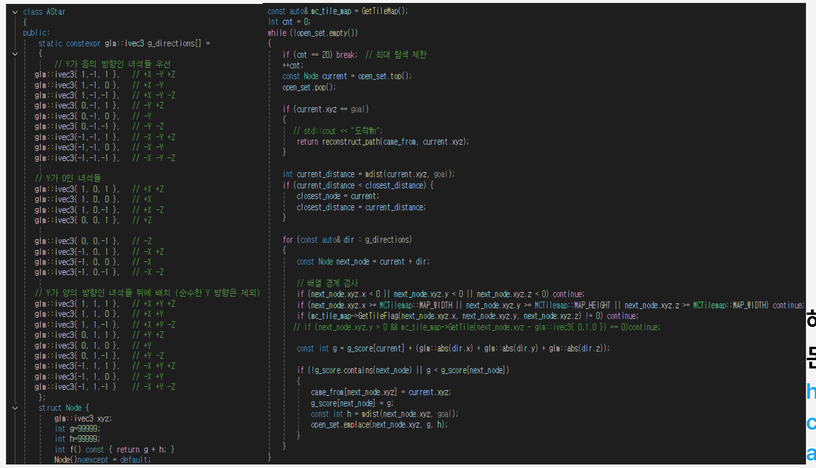
매크로를 이용한 프로토콜 생성

packet_define.h
Common\\PacketBase.hpp
Client\\CaveGame\\PacketBase.cpp
Server\\PacketBase.cpp
참조

네트워크 게임 프로그래밍 텀프로젝트 : 구현 내용

구현 내용

-3차원 공간에서의 A* 알고리즘 구현



```
class AStar
{
public:
    static constexpr glm::ivec3 g_directions[] =
    {
        // Y가 같의 방향인 녀석들 우선
    glm::ivec3( 1,-1, 1 ),  // +X -Y +Z
    glm::ivec3( 1,-1, 0 ),  // +X -Y
    glm::ivec3( 1,-1,-1 ),  // +X -Y -Z
    glm::ivec3( 0,-1, 1 ),  // -Y +Z
    glm::ivec3( 0,-1, 0 ),  // -Y
    glm::ivec3( 0,-1,-1 ),  // -Y -Z
    glm::ivec3(-1,-1, 1 ),  // -X -Y +Z
    glm::ivec3(-1,-1, 0 ),  // -X -Y
    glm::ivec3(-1,-1,-1 ),  // -X -Y -Z

    // Y가 0인 녀석들
    glm::ivec3( 1, 0, 1 ),  // +X +Z
    glm::ivec3( 1, 0, 0 ),  // +X
    glm::ivec3( 1, 0,-1 ),  // +X -Z
    glm::ivec3( 0, 0, 1 ),  // +Z

    glm::ivec3( 0, 0,-1 ),  // -Z
    glm::ivec3(-1, 0, 1 ),  // -X +Z
    glm::ivec3(-1, 0, 0 ),  // -X
    glm::ivec3(-1, 0,-1 ),  // -X -Z

    // Y가 양의 방향인 녀석들 뒤에 배치 (순수한 Y 방향은 제외)
    glm::ivec3( 1, 1, 1 ),  // +X +Y +Z
    glm::ivec3( 1, 1, 0 ),  // +X +Y
    glm::ivec3( 1, 1,-1 ),  // +X +Y -Z
    glm::ivec3( 0, 1, 1 ),  // +Y +Z
    glm::ivec3( 0, 1, 0 ),  // +Y
    glm::ivec3( 0, 1,-1 ),  // +Y -Z
    glm::ivec3(-1, 1, 1 ),  // -X +Y +Z
    glm::ivec3(-1, 1, 0 ),  // -X +Y
    glm::ivec3(-1, 1,-1 )   // -X +Y -Z
    };
    struct Node {
        glm::ivec3 xyz;
        int g=99999;
        int h=99999;
        int f() const { return g + h; }
        Node()noexcept = default;
```

```
const auto& ac_tile_map = GetTileMap();
int cnt = 0;
while (!open_set.empty())
{
    if (cnt == 20) break;  // 최대 탐색 제한
    ++cnt;
    const Node current = open_set.top();
    open_set.pop();

    if (current.xyz == goal)
    {
        // std::cout << "도착함";
        return reconstruct_path(came_from, current.xyz);
    }

    int current_distance = adist(current.xyz, goal);
    if (current_distance < closest_distance) {
        closest_node = current;
        closest_distance = current_distance;
    }

    for (const auto& dir : g_directions)
    {
        const Node next_node = current + dir;

        // 배열 경계 검사
        if (next_node.xyz.x < 0 || next_node.xyz.y < 0 || next_node.xyz.z < 0) continue;
        if (next_node.xyz.x >= MCTilemap::MAP_WIDTH || next_node.xyz.y >= MCTilemap::MAP_HEIGHT || next_node.xyz.z >= MCTilemap::MAP_WIDTH) continue;
        if (ac_tile_map->GetTileFlag(next_node.xyz.x, next_node.xyz.y, next_node.xyz.z) != 0) continue;
       // if (next_node.xyz.y > 0 && ac_tile_map->GetTile(next_node.xyz - glm::ivec3( 0,1,0 )) == 0)continue;

        const int g = g_score[current] + (glm::abs(dir.x) + glm::abs(dir.y) + glm::abs(dir.z));

        if (!g_score.contains(next_node) || g < g_score[next_node])
        {
            came_from[next_node.xyz] = current.xyz;
            g_score[next_node] = g;
            const int h = adist(next_node.xyz, goal);
            open_set.emplace(next_node.xyz, g, h);
        }
    }
}
```

AStar.h / cpp 문서 참고

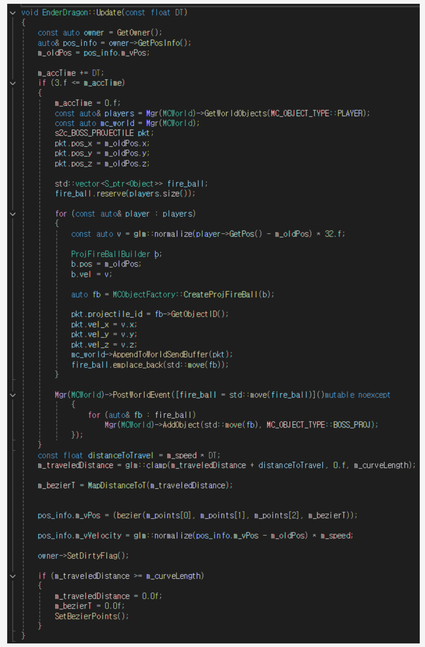
해당 알고리즘이 실제로 적용되어 몬스터가 움직이는 동영상 노션 링크:
https://lovely-nasturtium-c18.notion.site/147c647a121b80c2aa17f348888ef722?pvs=25

- 보스패턴 및 이동

```
void EnderDragon::SetBezierPoints() noexcept
{
    m_points[0] = GetOwner()->GetPos();
    const auto num = uid(dre);

    const auto b = m_randomPoints.data();
    m_points[1] = *(b + num);
    m_points[2] = *(b + (19 - num));
    const auto& players = Mgr(MCWorld)->GetWorldObjects(MC_OBJECT_TYPE::PLAYER);
    if (const auto num = (int)players.size())
    {
        const auto y = m_points[2].y;
        glm::vec3 temp{};
        auto b = players.data();
        const auto e = b + num;
        while (e != b) { temp += (*b++)->GetPos(); }
        temp /= num;
        m_points[2] = temp;
        m_points[2].y = y;
    }
    GenerateArcLengthTable(m_points[0], m_points[1], m_points[2]);
    m_curveLength = m_arcLengthTable.back().second;
}
```

보스 몬스터 작동 영상 노션 링크:
https://lovely-nasturtium-c18.notion.site/14ec647a121b80df8c0dd7fd73d9b2f7
https://lovely-nasturtium-c18.notion.site/14ec647a121b80a3b5eccf4a4073e93a

Server\\EnderDragon.cpp 참고



```
void EnderDragon::Update(const float DT)
{
    const auto owner = GetOwner();
    auto& pos_info = owner->GetPosInfo();
    m_oldPos = pos_info.m_vPos;

    m_accTime += DT;
    if (3.f <= m_accTime)
    {
        m_accTime = 0.f;
        const auto& players = Mgr(MCWorld)->GetWorldObjects(MC_OBJECT_TYPE::PLAYER);
        const auto mc_world = Mgr(MCWorld);
        s2c_BOSS_PROJECTILE pkt;
        pkt.pos_x = m_oldPos.x;
        pkt.pos_y = m_oldPos.y;
        pkt.pos_z = m_oldPos.z;

        std::vector<S_ptr<Object>> fire_ball;
        fire_ball.reserve(players.size());

        for (const auto& player : players)
        {
            const auto v = glm::normalize(player->GetPos() - m_oldPos) * 32.f;

            ProjFireBallBuilder b;
            b.pos = m_oldPos;
            b.vel = v;

            auto fb = MCObjectFactory::CreateProjFireBall(b);

            pkt.projectile_id = fb->GetObjectID();
            pkt.vel_x = v.x;
            pkt.vel_y = v.y;
            pkt.vel_z = v.z;
            mc_world->AppendToWorldSendBuffer(pkt);
            fire_ball.emplace_back(std::move(fb));
        }

        Mgr(MCWorld)->PostWorldEvent([fire_ball = std::move(fire_ball)]()mutable noexcept
            {
                for (auto& fb : fire_ball)
                    Mgr(MCWorld)->AddObject(std::move(fb), MC_OBJECT_TYPE::BOSS_PROJ);
            });
    }
    const float distanceToTravel = m_speed * DT;
    m_traveledDistance = glm::clamp(m_traveledDistance + distanceToTravel, 0.f, m_curveLength);

    m_bezierT = MapDistanceToT(m_traveledDistance);


    pos_info.m_vPos = (bezier(m_points[0], m_points[1], m_points[2], m_bezierT));

    pos_info.m_vVelocity = glm::normalize(pos_info.m_vPos - m_oldPos) * m_speed;

    owner->SetDirtyFlag();

    if (m_traveledDistance >= m_curveLength)
    {
        m_traveledDistance = 0.0f;
        m_bezierT = 0.0f;
        SetBezierPoints();
    }
}
```

- 투사체 동기화 및 충돌판정과 사망판정

```
void Projectile::CheckGroupCollision(Projectile* const proj, const MC_OBJECT_TYPE type, const float radius, const float height, const int dmg) noexcept
{
    const auto& target_group = Mgr(MCWorld)->GetWorldObjects(type);
    const auto owner = proj->GetOwner();
    const auto project_pos = owner->GetPos();
    const float collision_radius = radius;
    const float collision_height = height;

    for (const auto& target : target_group)
    {
        const auto& target_pos = target->GetPos();
        const bool is_hit = glm::distance(glm::vec2(target_pos.x, target_pos.z), glm::vec2(project_pos.x, project_pos.z)) <= collision_radius
            && project_pos.y >= target_pos.y && project_pos.y <= target_pos.y + collision_height;
        if (is_hit)
        {
            //  std::cout << "충돌!\n";
            if (target->GetObjectType() == MC_OBJECT_TYPE::MONSTER)
            {
                auto& v = target->GetPosInfo().m_vVelocity;
                v.y = 20.f;
            }
            target->DecHP(dmg);
            owner->SetInvalid();
            return;
        }
    }
}
```

Projectile.h / cpp 참고

화살 투사체 동기화 및 충돌판정과 사망 판정 구현 작동 영상 노션 링크:

VS 몬스터
https://lovely-nasturtium-c18.notion.site/14cc647a121b80a6b676de48ace74437

VS 플레이어
https://lovely-nasturtium-c18.notion.site/14dc647a121b804f9785c0138bd66545?pvs=25

- 기타 구현 내용

1. FSM기반 몬스터 AI
Server\\FSM.cpp , Server\\State.cpp 참조

2. 드랍 아이템 등장 및 획득
Server\\DropItem.cpp 참조

3. HP 및 사망 시스템
Server\\HP.cpp 참조

4. 게임스레드와 IO스레드가 통신하기 위한 Lock-Free 큐 제작
Server\\LFQueue.hpp 참조

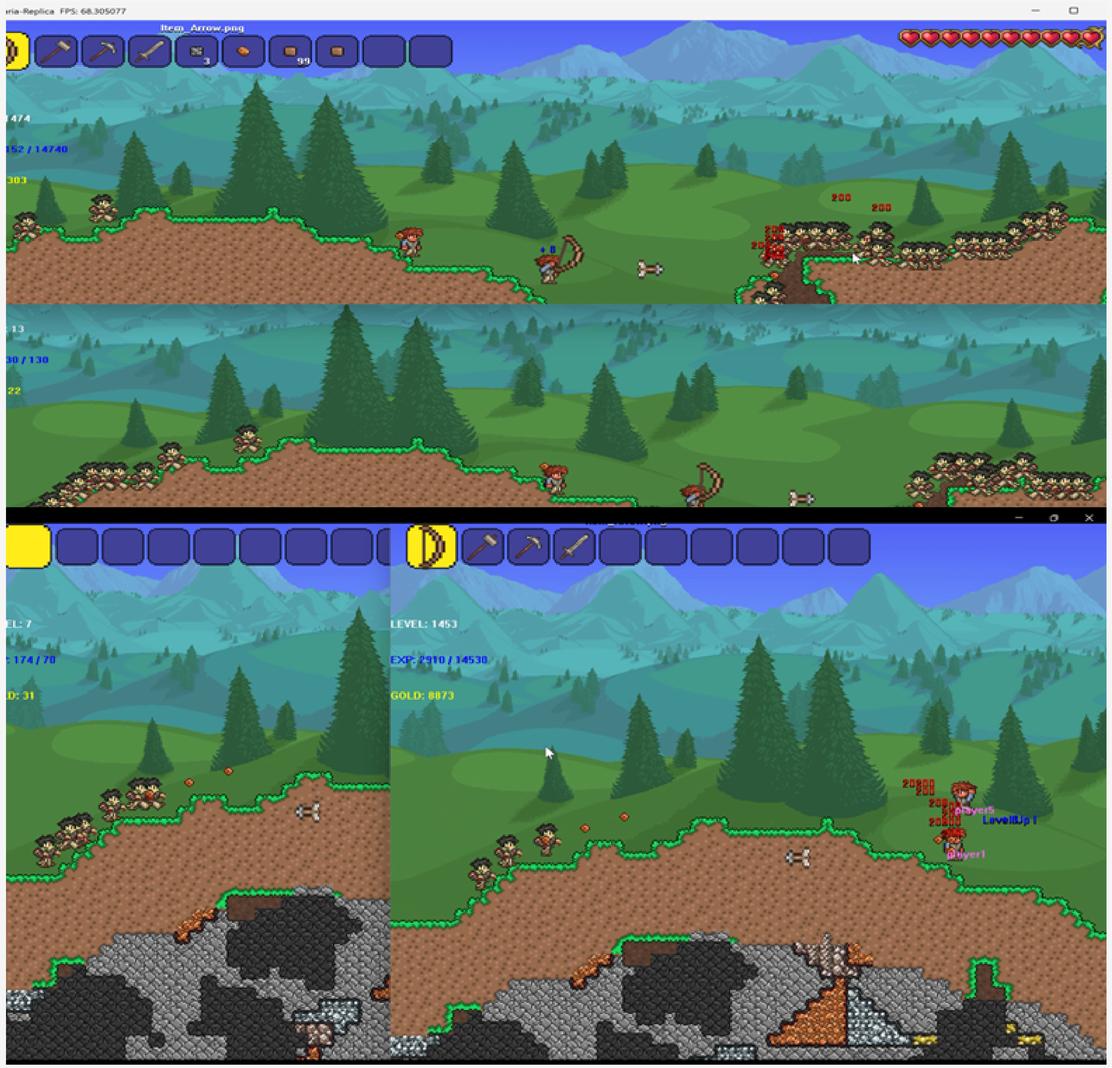
5. 클라이언트 사이드에서 서버로부터 전달되는 유저·몬스터·각종 컨텐츠 정보 동기화 작업
Client\\CaveGame\\ServerObjectManager.h , Client\\CaveGame\\ServerObjectManager.cpp 참조

6. OpenGL을 이용한 클라이언트 게임 프레임워크
- Client 폴더의 MyOpenGL 프로젝트

기타 프로젝트 : 테라리아 모작

게임명	Terraria-Replica(테라리아 모작)
게임 장르	사이드 뷰 2D 어드벤처 샌드박스 게임
개발 기간	2023.05 ~ 2023.09
개발 인원	2명
사용 기술	C++20, Win32API
역할 및 구현 내용	- Win32API 기반 게임프레임 워크 제작 - 배경 레이어 멀티스레드 렌더링 시스템 - 멀티스레드를 이용한 로딩화면 렌더링 및 실제 로딩 병렬처리 구현 - 이미지 회전이 가능한 2D 애니메이션 구조 - 코루틴을 이용한 카메라 워킹 - 화면 암전 시스템 - 넉백 및 카메라 흔들림
Git	https://github.com/hypernagox/TermProjectGameFrameWork
시연영상	https://www.youtube.com/watch?v=_183wB4UNio
특징	한국공학대학교 게임공학과 과제전 윈도우프로그래밍 분야 2등

기타 프로젝트 : 테라리아 모작 + 서버연동



게임명	Terraria-Replica IOCP (테라리아 모작 + 서버연동)
게임 장르	사이드 뷰 2D 어드벤처 샌드박스 게임
개발 기간	2024.05 ~ 2024.07
개발 인원	1명
사용 기술	C++20, Win32API, IOCP, DB Query Server
역할 및 구현 내용	- 앞서 제작했던 Terraria-Replica를 멀티플레이가 가능하도록 IOCP 서버 연동 - Protocol Buffers라이브러리 사용 및 자동화 툴 제작 - A*를 이용한 몬스터의 길 찾기 알고리즘 적용 - 원거리 투사체 충돌판정을 포함한 전투 시스템 - 메이플스토리의 포탈 시스템을 참고한 맵 구조 및 이동 - 드랍 아이템 및 획득/사용 로직 - 장비 아이템 변경 및 사용 로직 - 애니메이션 동기화 작업 - DB Query Server 별도 제작 - 경험치를 통한 성장 시스템 및 DB연동 - DB를 통한 사용자 정보 관리
Git	https://github.com/hypernagox/TUKGameServerTermProject
시연영상	https://youtu.be/_IY99Dzd5X0

읽어주셔서 감사합니다

Thank you

최 재 혁

연락처 | 010.4787.5323 **메일** | hypernagox@tukorea.ac.kr